



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

Chương 3: Công nghệ thành phần với JavaBeans và EJB

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

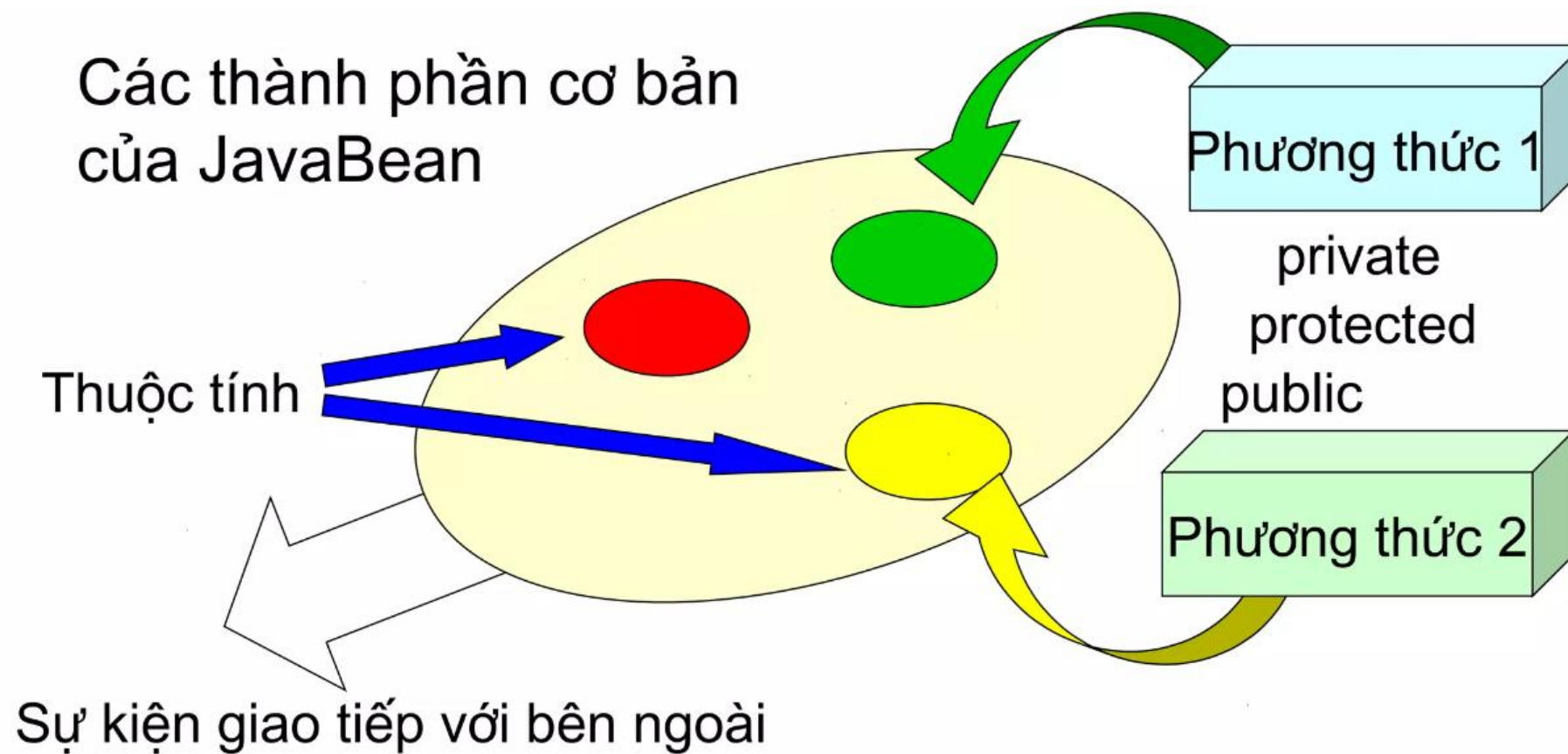
Nội dung của chương này tập trung xem xét một số chủ đề sau đây:

- Công nghệ thành phần với JavaBeans
- J2EE và kiến trúc EJB
- Các khái niệm về thành phần EJB và môi trường chạy của nó
- Mô hình thành phần của EJB
- Mô hình kết nối của EJB
- Hướng dẫn xây dựng, triển khai và sử dụng thành phần EJB

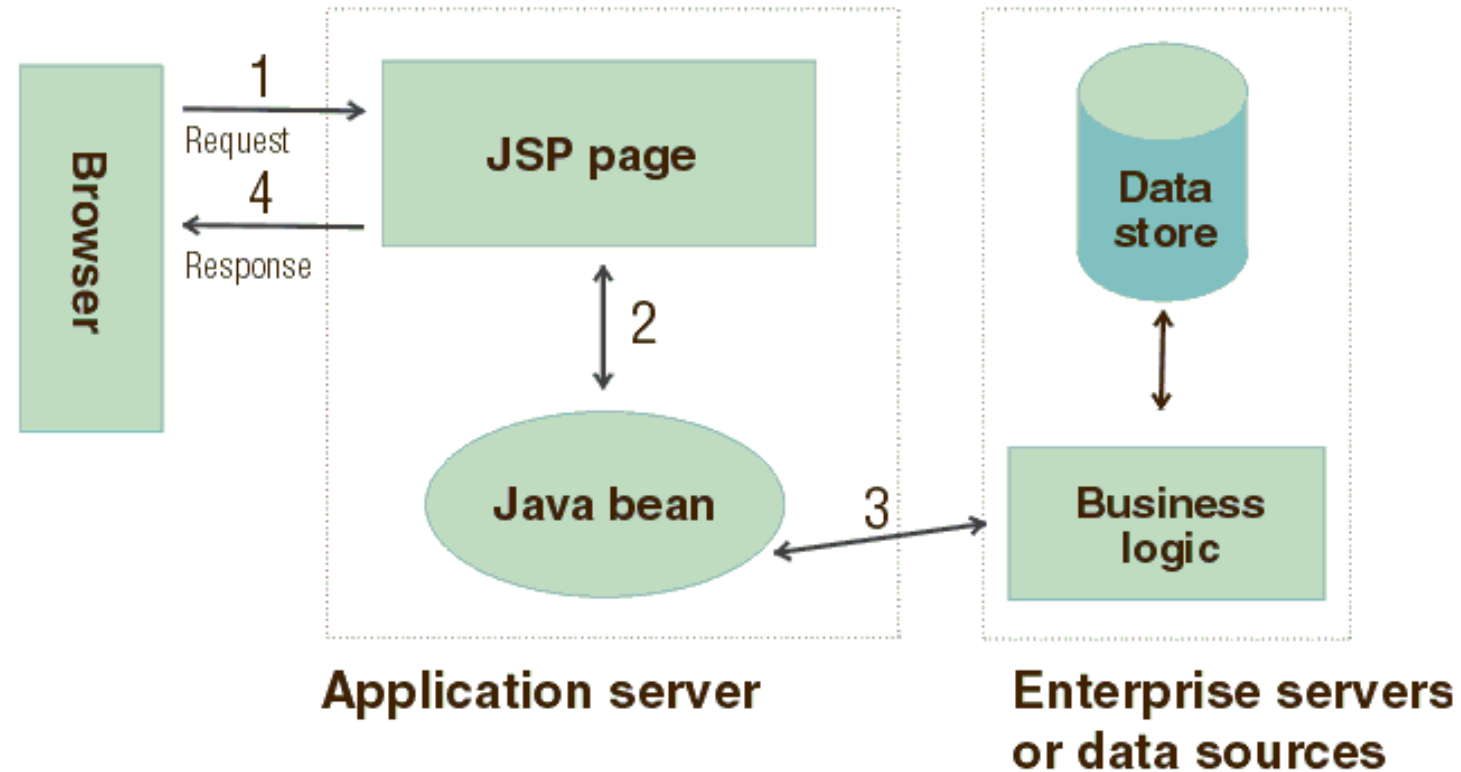
| 3.1 CÔNG NGHỆ THÀNH PHẦN VỚI JAVABEANS

- **JavaBeans** chính là các lớp thuần Java được sử dụng để thực hiện các thực thể trong chương trình Java.
- Một lớp JavaBeans sẽ có 3 đặc điểm như sau về mặt cú pháp:
 - Là một public class.
 - Có các thuộc tính private.
 - Ứng với mỗi thuộc tính, lớp sẽ cung cấp một cặp phương thức setter/getter để truy cập và thao tác với giá trị của từng thuộc tính.
- **Đặc trưng của JavaBeans:**
 - Javabeans cung cấp một constructor mặc định, không có tham số.
 - Nó có thể xếp thứ tự và triển khai Serializable interface (*cơ chế để ghi trạng thái của một đối tượng vào một byte stream để truyền qua mạng*).
 - Nó có thể có một số đặc tính mà có thể được đọc và được viết.
 - Nó có thể có một số phương thức “getter” và “setter” cho các đặc tính.

| Thành phần của JavaBean



| JavaBeans trong công nghệ JSP



Ví dụ JavaBeans

Các đặc tính JavaBeans được truy cập thông qua hai phương thức.

- `getProperty()`
- `setProperty()`

```
package com.tutorialspoint;

public class StudentsBean implements java.io.Serializable
{
    private String firstName = null;
    private String lastName = null;
    private int age = 0;

    public StudentsBean() {
    }

    public String getFirstName(){
        return firstName;
    }

    public String getLastName(){
        return lastName;
    }

    public int getAge(){
        return age;
    }

    public void setFirstName(String firstName){
        this.firstName = firstName;
    }

    public void setLastName(String lastName){
        this.lastName = lastName;
    }

    public void setAge(Integer age){
        this.age = age;
    }
}
```

| Truy cập JavaBeans

- Khai báo JavaBeans sử dụng trong JPS bằng `useBean`. Sau khi khai báo, Bean này trở thành biến scripting có thể truy cập cả các phần tử Scripting và các Custom Tags khác trong JSP.
- Sử dụng `<jsp:getProperty/>` để truy cập các phương thức get và `<jsp:setProperty/>` để truy cập các phương thức set. Cú pháp:

`<jsp:useBean id="bean's name" scope="bean's scope" typeSpec/>`

- Các giá trị cho thuộc tính scope có thể là page, request, session hoặc application tùy theo yêu cầu.

- Ví dụ:

```
<jsp:useBean id="id" class="bean's class" scope="bean's scope">
  <jsp:setProperty name="bean's id" property="property name"
    value="value"/>
  <jsp:getProperty name="bean's id" property="property name"/>
  .....
</jsp:useBean>
```

| Truy cập JavaBeans

```
<html>
<head>
<title>get and set properties Example</title>
</head>
<body>

<jsp:useBean id="students"
              class="com.tutorialspoint.StudentsBean">
  <jsp:setProperty name="students" property="firstName"
                  value="Zara"/>
  <jsp:setProperty name="students" property="lastName"
                  value="Ali"/>
  <jsp:setProperty name="students" property="age"
                  value="10"/>
</jsp:useBean>

<p>Student First Name:
  <jsp:getProperty name="students" property="firstName"/>
</p>
<p>Student Last Name:
  <jsp:getProperty name="students" property="lastName"/>
</p>
<p>Student Age:
  <jsp:getProperty name="students" property="age"/>
</p>

</body>
</html>
```



```
Student First Name: Zara
Student Last Name: Ali
Student Age: 10
```


| 3.2 KIẾN TRÚC EJB

- Tổng quan về kiến trúc EJB và J2EE platform:
 - **Enterprise Java Beans** (EJB) do Sun Microsystems Inc công bố vào năm 1998, EJB là kiến trúc thành phần dành cho việc phát triển và triển khai các ứng dụng phân tán hướng thành phần.
 - Một thành phần **EJB** có khả năng sử dụng lại, viết một lần chạy mọi nơi **WORA** (Writing Once, Run Anywhere); có khả năng di động, được biên dịch để có thể triển khai trên bất kỳ server nào như J2EE, Struts, WebLogic.
 - EJB cung cấp một tập API và các dịch vụ khác nhau để người phát triển tập trung vào logic nghiệp vụ.

| Các phiên bản của EJB

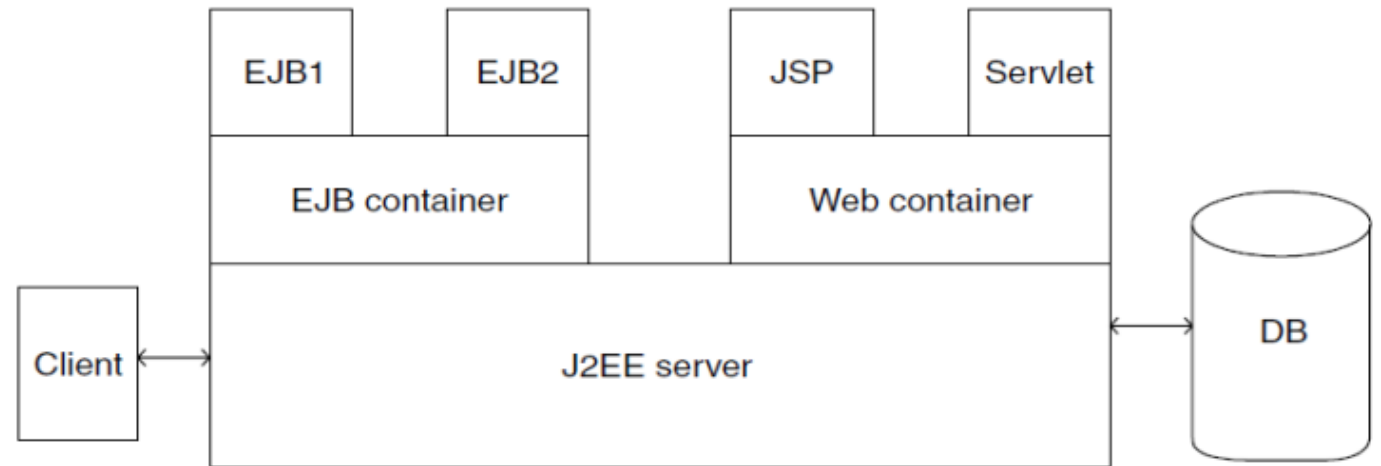
- EJB 1.0 (1998-03-24)
- EJB 1.1 (1999-12-17)
- EJB 2.0 (2001-08-22)
- EJB 2.1 (2003-11-24)
- EJB 3.0 (2006-05-11)
- EJB 3.1 (2009-12-10)
- EJB 3.2 (2013-05-28)
- EJB 3.2.6 (2019-08-23)
- EJB 4.0 (2020-05-22)

| KIẾN TRÚC EJB

- Một số đặc trưng của EJB:
 - Một thành phần EJB có thể được phát triển một lần và sau đó có thể sử dụng lại trong nhiều ứng dụng và triển khai trên nhiều nền tảng khác nhau mà không phải biên dịch và sửa lại mã nguồn.
 - Một thành phần EJB là một thành phần phía server cung cấp các dịch vụ cho điều khiển từ xa hoặc client cục bộ, trong khi một java bean là một thành phần phía client được cài đặt và chạy hầu hết ở phía client.
 - Một thành phần EJB được chứa bởi container của nó và container được hỗ trợ bởi J2EE hoặc bất kỳ công cụ tuân theo đặc tả J2EE.

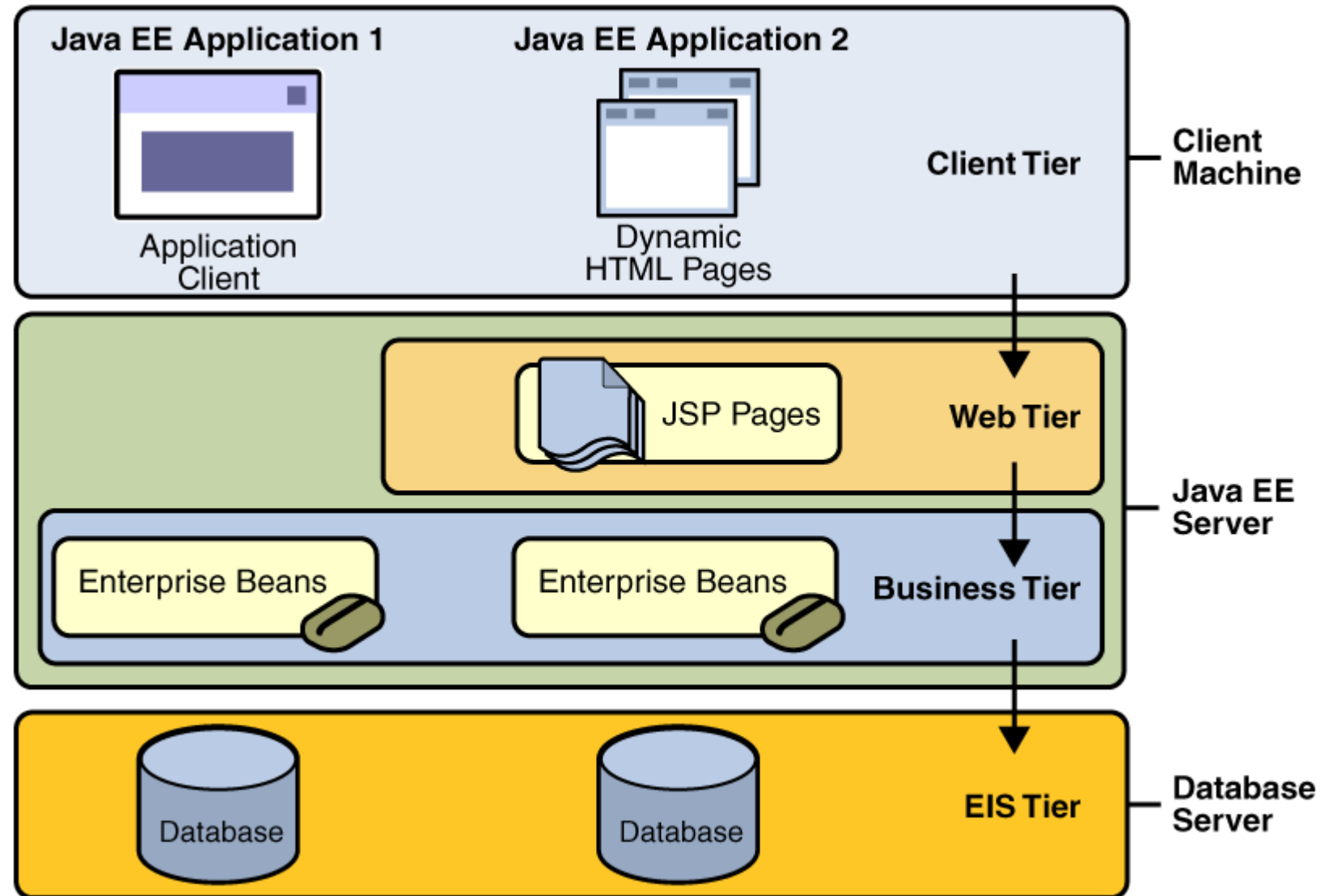
| J2EE Server

- J2EE (Java 2 Platform Enterprise Edition) là phiên bản Java dành cho doanh nghiệp, giúp cho việc lập trình ứng dụng được đơn giản hóa và giảm thiểu lập trình bằng cách tạo các thành phần module, có thể tái sử dụng và tiêu chuẩn hóa.
- J2EE server cung cấp một số dịch vụ sau:
 - JNDI (Java Naming and Directory Interface) API cho phép các client tìm kiếm và xác định vị trí container chứa các thành phần EJB đã đăng ký.
 - Hỗ trợ việc xác thực và an toàn.
 - Hỗ trợ dịch vụ HTTP và các dịch vụ liên quan đến EJB, các dịch vụ đa luồng...



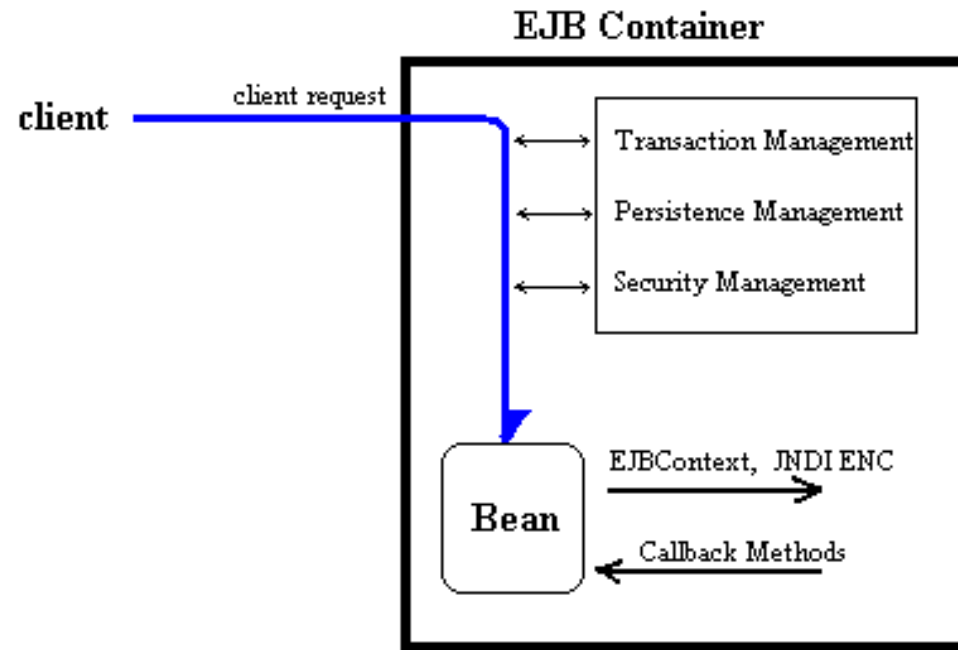
Kiến trúc J2EE

| J2EE Server



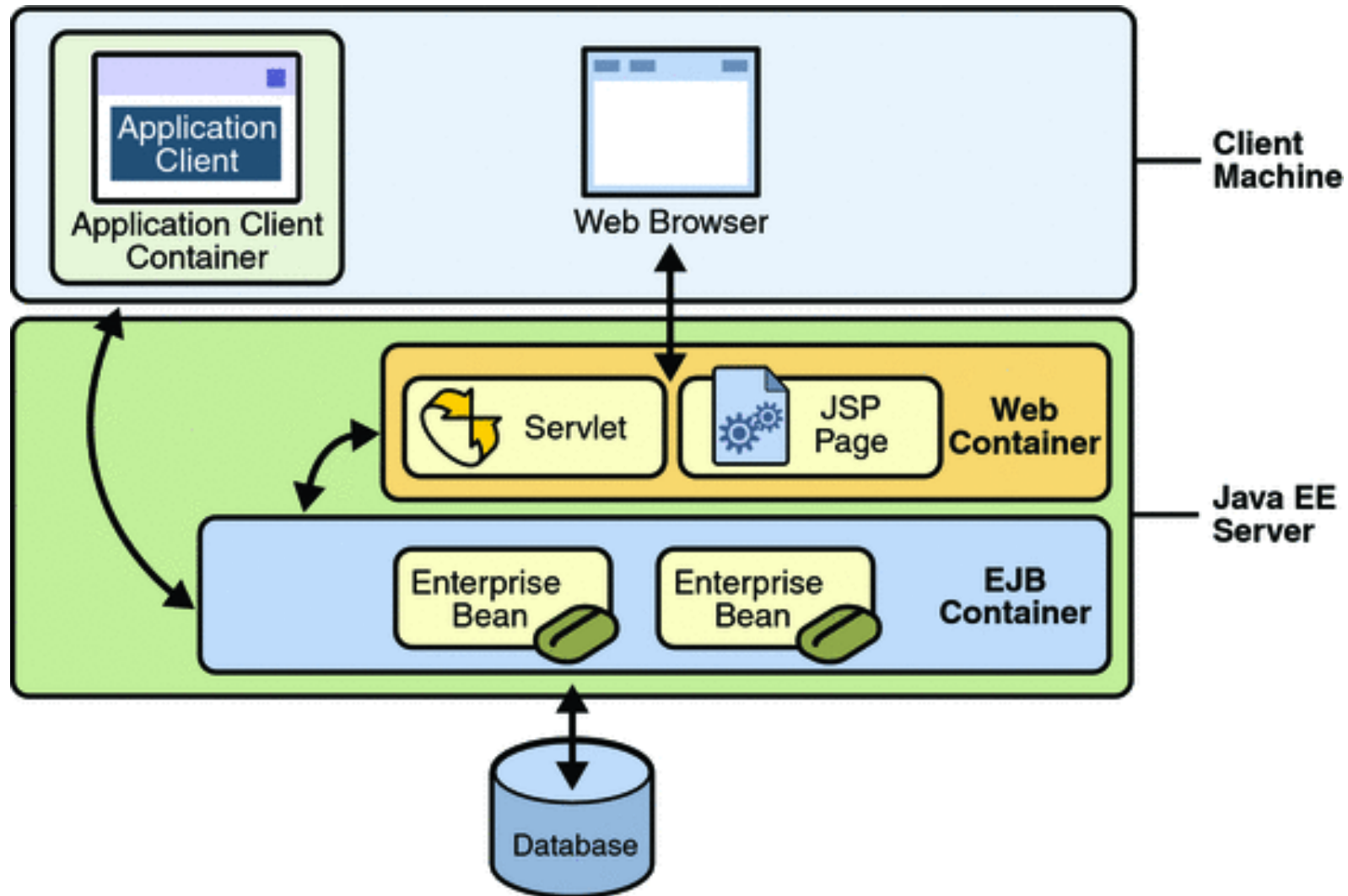
| Container

- Một thể hiện của EJB chạy trên một **EJB container**. **Container** là môi trường chạy (tập các file class được sinh ra trong quá trình phát triển) điều khiển một thể hiện của thành phần EJB và cung cấp tất cả các dịch vụ quản lý cần thiết cho toàn bộ vòng đời của nó:
 - Quản lý giao dịch và an toàn (transaction, security): đảm bảo các đặc tính giao dịch và an toàn của các thực thi giao dịch phân tán.
 - Quản lý lưu trữ (persistence): đảm bảo trạng thái của một bean thực thể (entity bean) được sao lưu bởi cơ sở dữ liệu.
 - Quản lý vòng đời (life cycle): đảm bảo sự dịch chuyển trạng thái của EJB trong vòng đời của nó



**EJB Containers manage
enterprise beans at runtime**

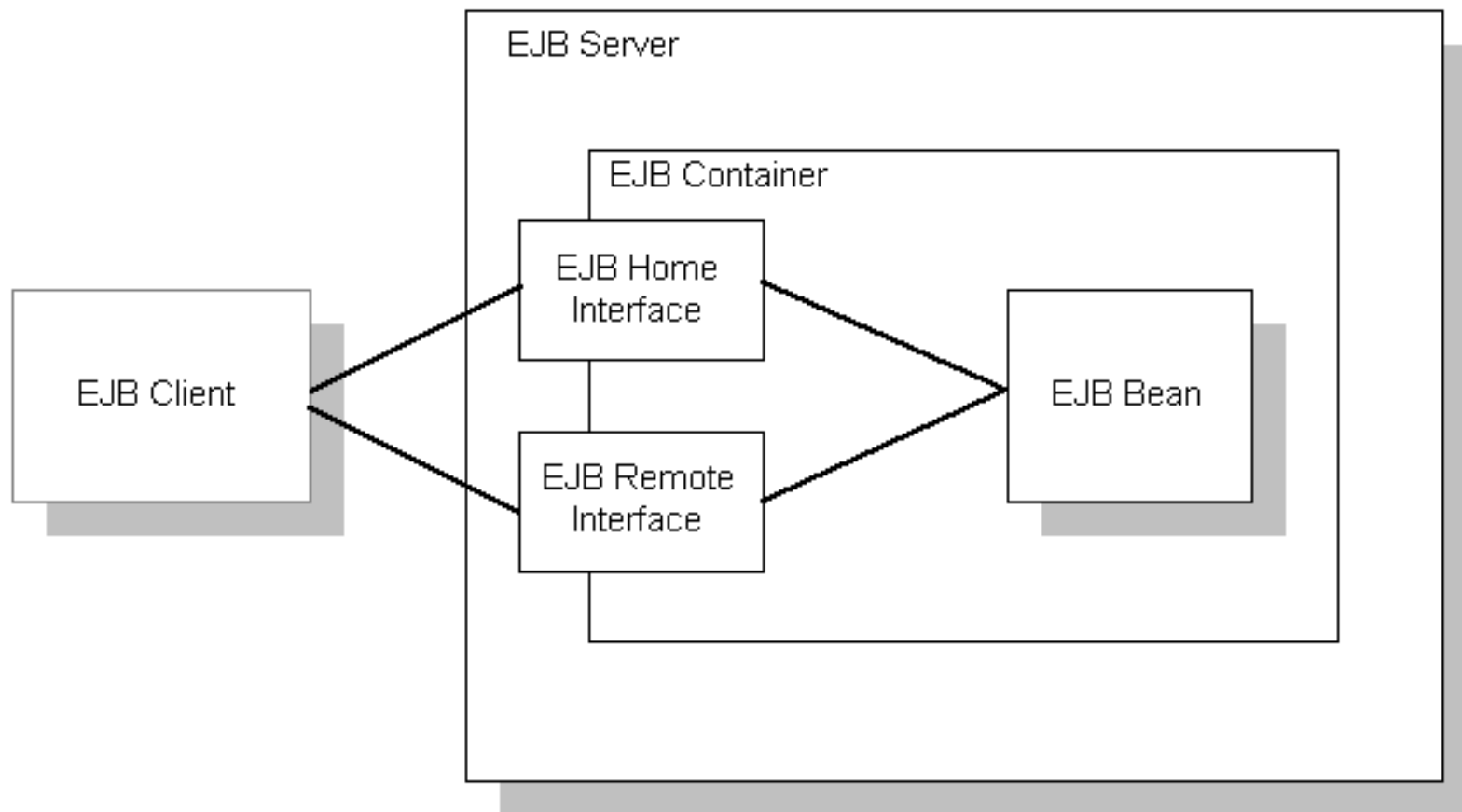
| J2EE Server và Container



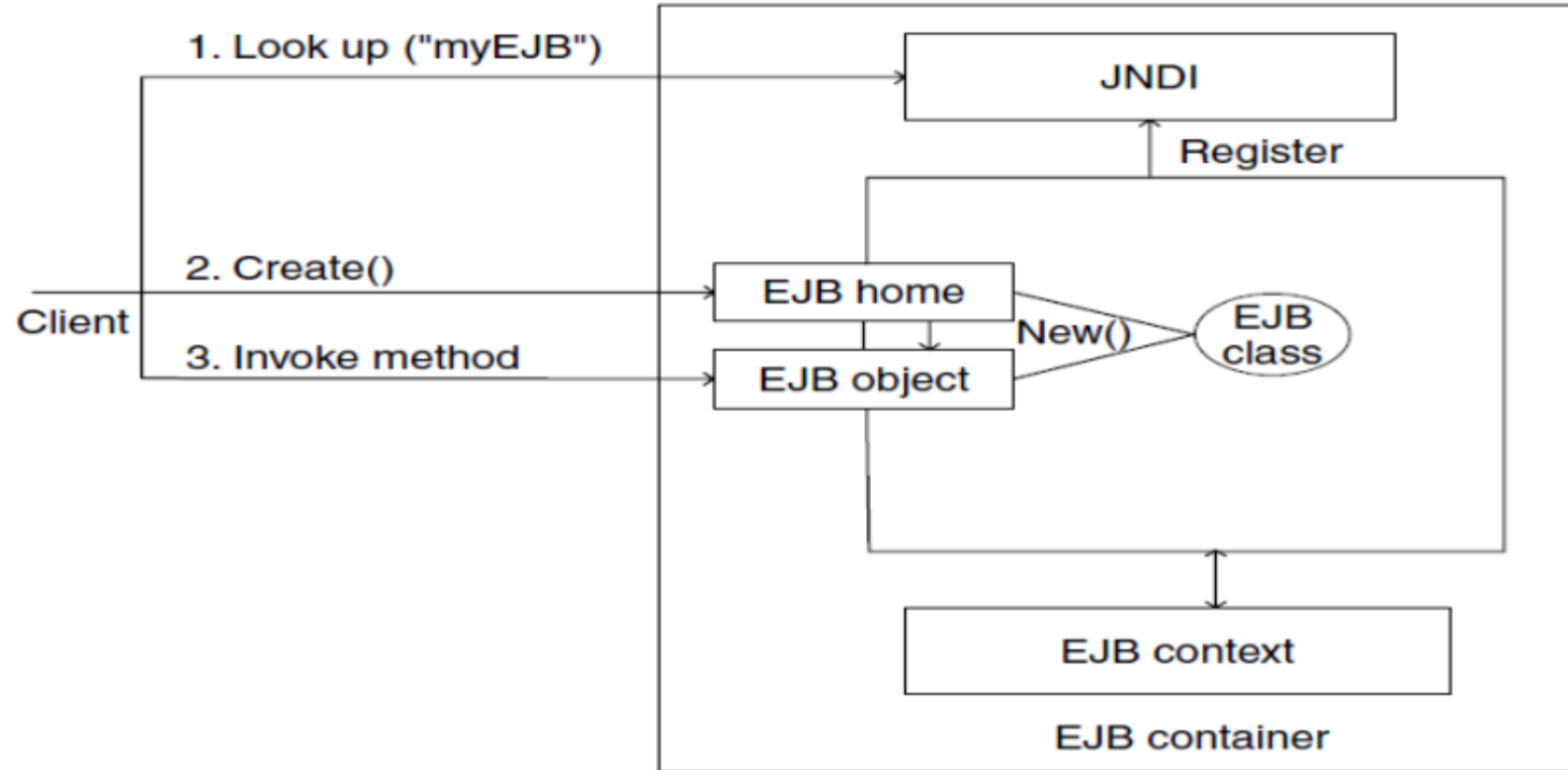
I 3.2 MÔ HÌNH THÀNH PHẦN CỦA EJB

- Mỗi thành phần EJB có một giao diện logic nghiệp vụ được tạo ra bởi thành phần đó nên các client có thể truy cập vào các thao tác logic nghiệp vụ thông qua giao diện này mà không cần phải biết cài đặt chi tiết đằng sau giao diện đó.
- Một EJB có những đặc điểm sau:
 - Chứa các business logic để thao tác với dữ liệu.
 - Được tạo ra và được quản lý bởi một container.
 - Xử lý các truy cập của client.
 - Một thành phần EJB là một hộp đen nghĩa là client của nó chỉ biết tương tác với thành phần nào chứ không biết cách hoạt động của nó.
 - Chứa metadata, như là thuộc tính của transaction và security, riêng biệt với các bean.
 - Cung cấp dịch vụ quản lý transaction, quản lý trạng thái, vòng đời và bảo mật.

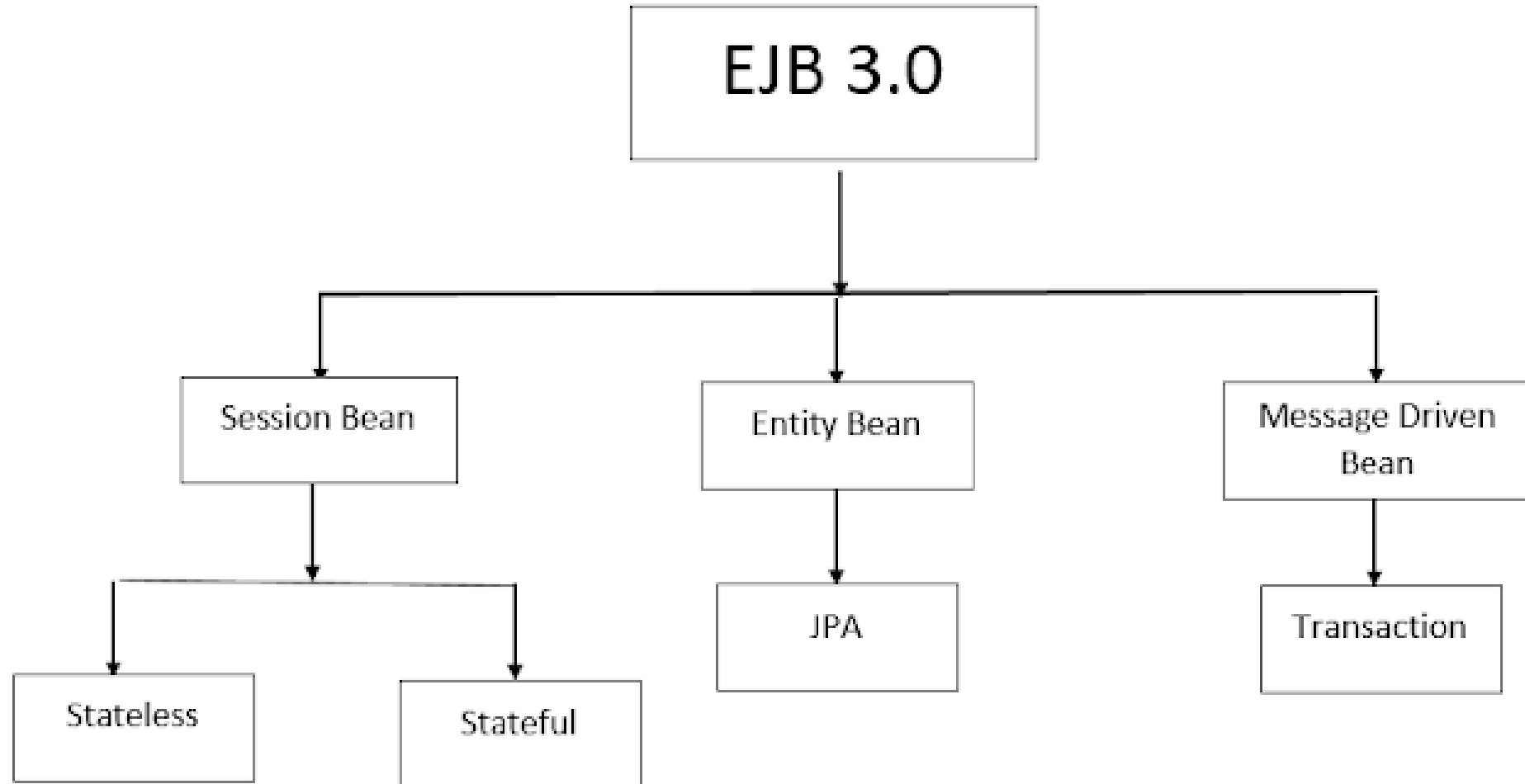
| KIẾN TRÚC EJB



I TƯƠNG TÁC GIỮA CLIENT VÀ THÀNH PHẦN CỦA EJB



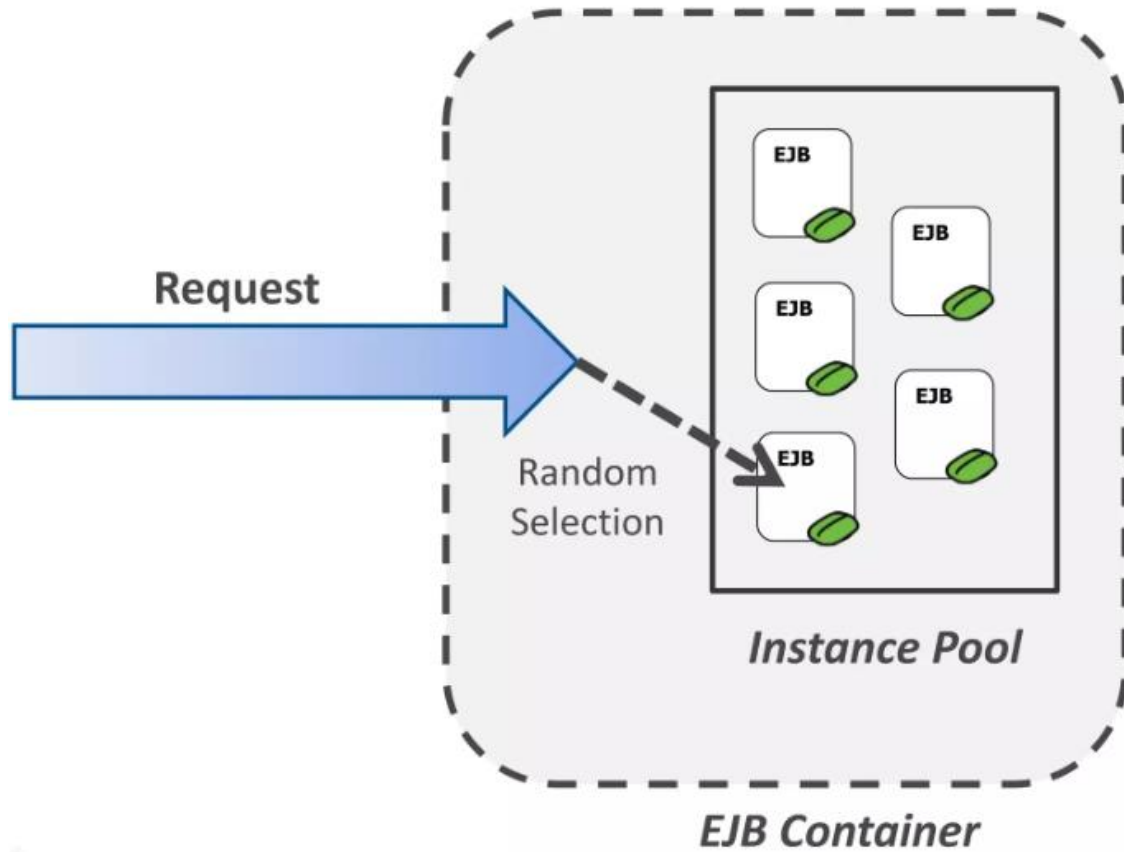
| Các loại EJB



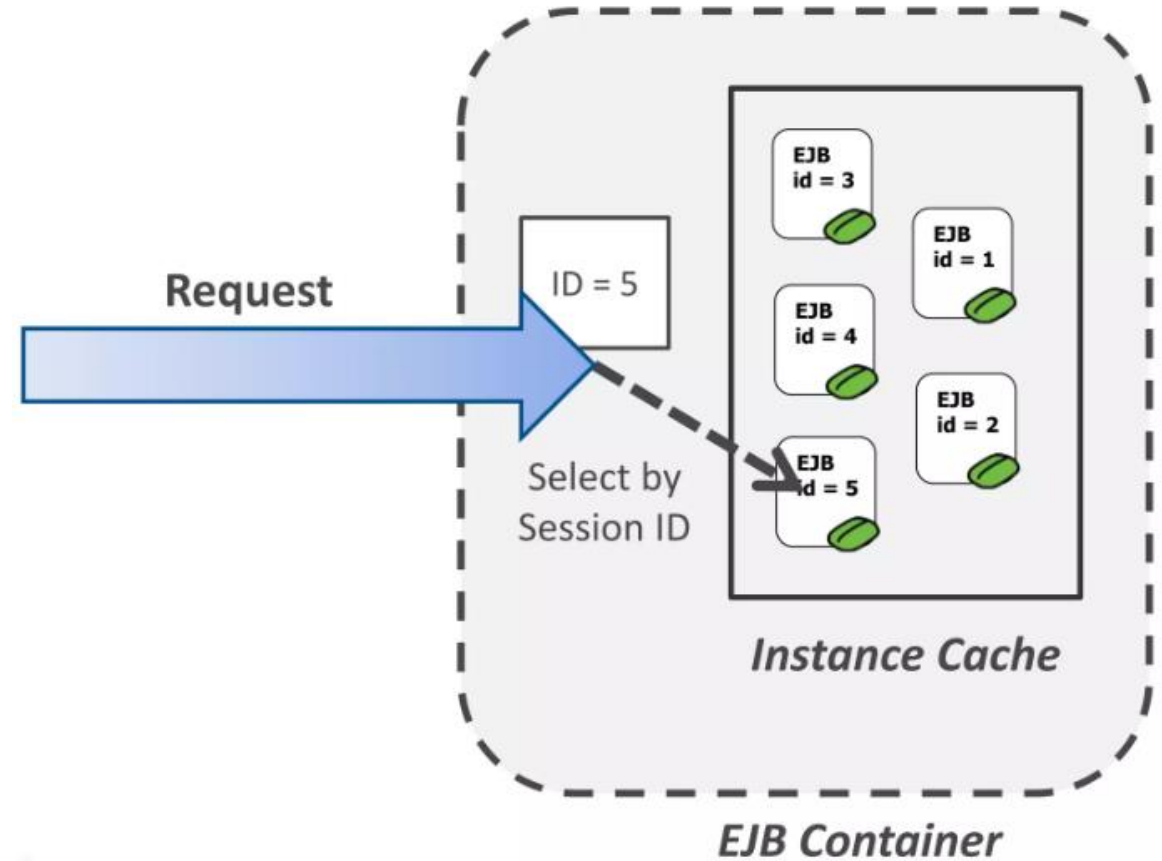
| Session Bean

- **Session Bean**: quản lý các nhiệm vụ tác nghiệp của các client của EJB. Client gửi một message tới session bean thông qua môi trường mạng, Session bean có thể triệu gọi tới một entity bean hay một session bean khác để hoàn thành nhiệm vụ mà client yêu cầu.
- Session bean có 3 loại:
 - **Stateless**: trạng thái các giao dịch của client sẽ không được lưu lại cho những lần giao dịch sau. Những lần giao dịch khác nhau sẽ hoàn toàn độc lập với nhau.
 - **Statefull**: trạng thái giao dịch tác vụ của client sẽ được lưu trữ lại để phục vụ cho những lần giao dịch sau.
 - **Singleton**: khởi tạo 1 lần và tồn tại suốt vòng đời của ứng dụng.

| Session Bean



Stateless Session Bean



Statefull Session Bean

| Ví dụ với Stateless Session Bean

```
@Local
public interface CalculationLocal {

    int sum(int a, int b);
}
```

```
@Stateless
public class CalculationBean implements CalculationLocal {

    public int sum(int a, int b) {
        return a + b;
    }
}
```

| Stateless Session Bean với Web Service

```
Package ejb.service;  
import javax.ejb.Stateless;  
import javax.jws.WebMethod;  
import javax.jws.WebService;  
@Stateless  
@WebService  
public class HelloServiceBean {  
    private String message = "Hello, ";  
    public void HelloServiceBean() {  
    }  
    @WebMethod  
    public String sayHello(String name) {  
        return message + name + ".";  
    }  
}
```

| Ví dụ với Statefull Session Bean

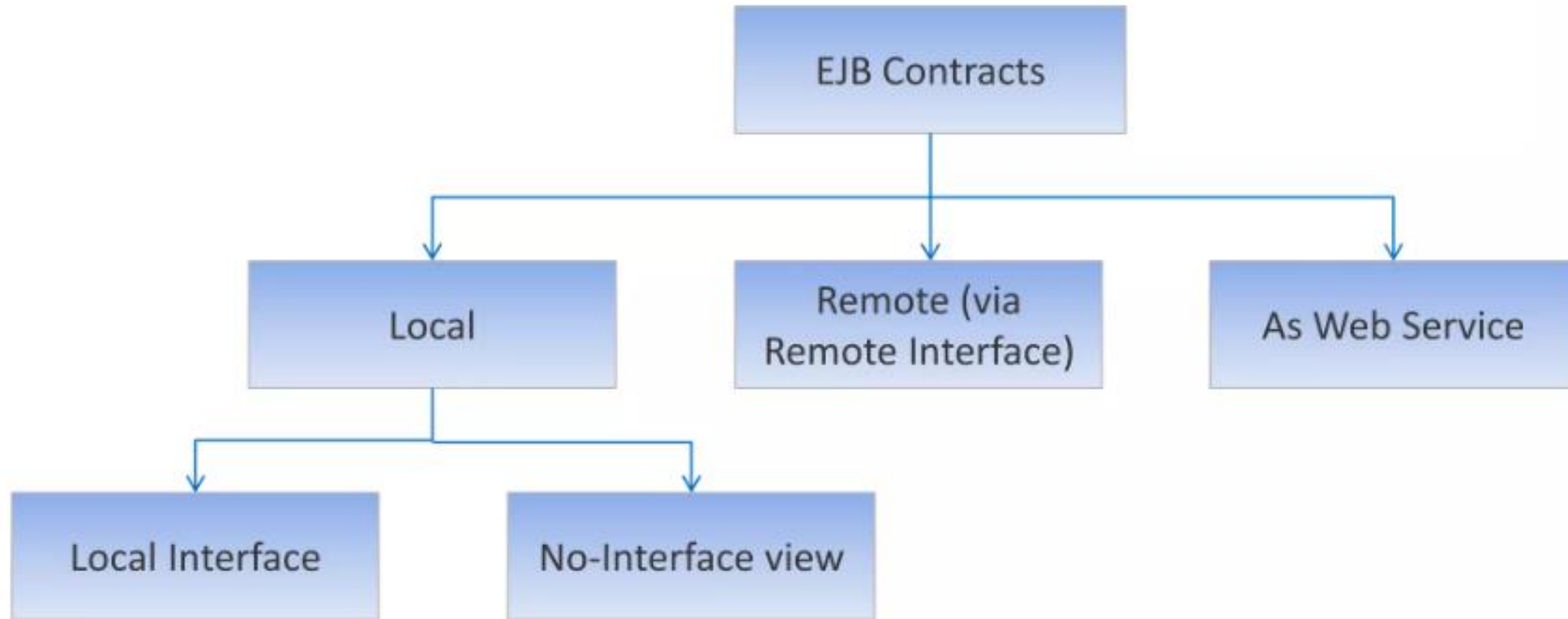
```
@Stateful
public class CalculationBean implements CalculationRemote {
    private int result = 0;
    @PostConstruct
    void construct() {}
    @PreDestroy
    void destroy() {}
    @PostActivate
    void activate() {}
    @PrePassivate
    void passivate() {}

    @Remove
    public void remove() {}

    public void add(int a) {
        result += a;
    }

    public int getResult() {
        return result;
    }
}
```


| Truy cập Session Bean



| Khai báo Bean Local

```
@Local public interface CalculationLocal { ... }
```

```
@Stateless
```

```
public class CalculationBean implements CalculationLocal { ... }
```

or

```
public interface CalculationLocal { ... }
```

```
@Stateless
```

```
@Local(CalculationLocal.class)
```

```
public class CalculationBean implements CalculationLocal { ... }
```

| Khai báo Bean Remote

- `@Remote`
`public interface` CalculationRemote {...}

`@Stateless`
`public class` CalculationBean
 `implements` CalculationRemote {...}

- Or decorate the bean class with `@Remote`:

```
public interface CalculationRemote {...}  
  
@Remote(CalculationRemote.class)  
public class CalculationBean  
                    implements CalculationRemote {...}
```

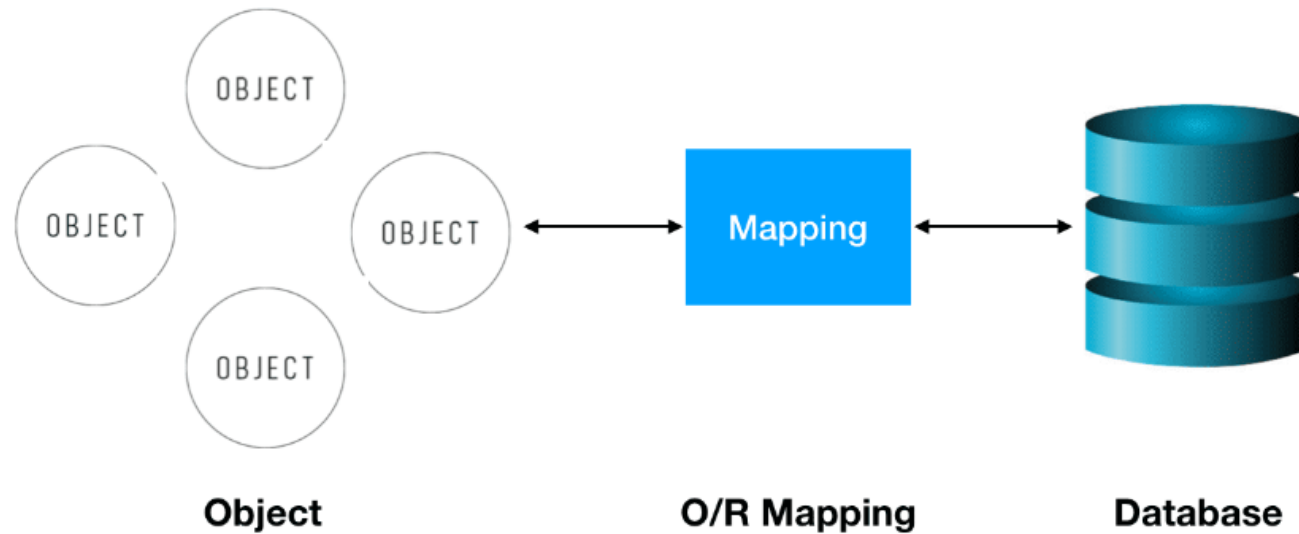
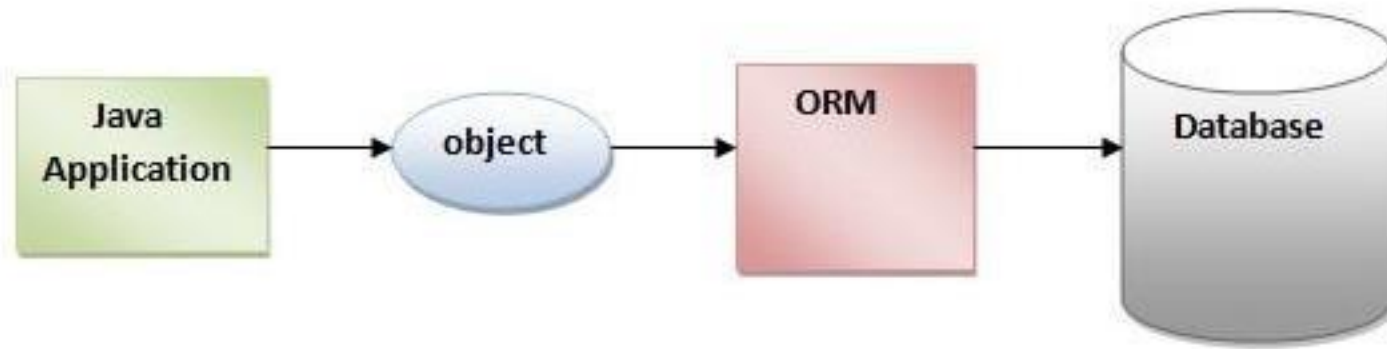
| Entity Bean, Message Driven Bean

- **Entity Bean**: chứa các thông tin tác vụ và các phương thức hoạt động của nó. Một entity bean có thể được lưu trữ và khôi phục lại một cách tự động bởi trình quản lý EJB, với việc sử dụng cơ sở dữ liệu quan hệ hay một hệ thống lưu trữ nào đó do bạn chọn.
 - Với EJB 2.x, có hai loại entity bean: BMP (entity bean Bean Managed Persistence) và CMP (Container Managed Persistence)
 - Với EJB 3.x có thể sử dụng JPA (Java Persistent API) để tạo ra các thực thể ánh xạ vào các bảng trong cơ sở dữ liệu.
- **Message Driven Bean**: chịu trách nhiệm điều khiển các message giữa client và server. Message này sẽ được truyền theo hình thức P2P hay broadcast.

| Java Persistence API

- JPA là một đặc tả Java cho việc ánh xạ giữa các đối tượng Java với cơ sở dữ liệu quan hệ sử dụng công nghệ phổ biến là ORM (Object Relational Mapping).
 - JPA cung cấp đầy đủ các công cụ cho phép chúng ta có thể thao tác với cơ sở dữ liệu một cách đơn giản và nhanh chóng.
 - JPA hoạt động như một cầu nối giữa các table/các mối quan hệ giữa các table trong database và các class/ mối quan hệ giữa các object. Ví dụ: table USER với các column (Id, username, password) sẽ tương ứng với class User.java với các field Id, username, password.
- > Từ đó mỗi khi truy vấn table hay các column ta sẽ gọi trực tiếp các phương thức trên các class, các field của class mà không cần quan tâm tới việc đang dùng loại database nào, kiểu dữ liệu database ra sao, ...

| Java Persistence API



| Java Persistence API

- **Lợi ích của JPA:**

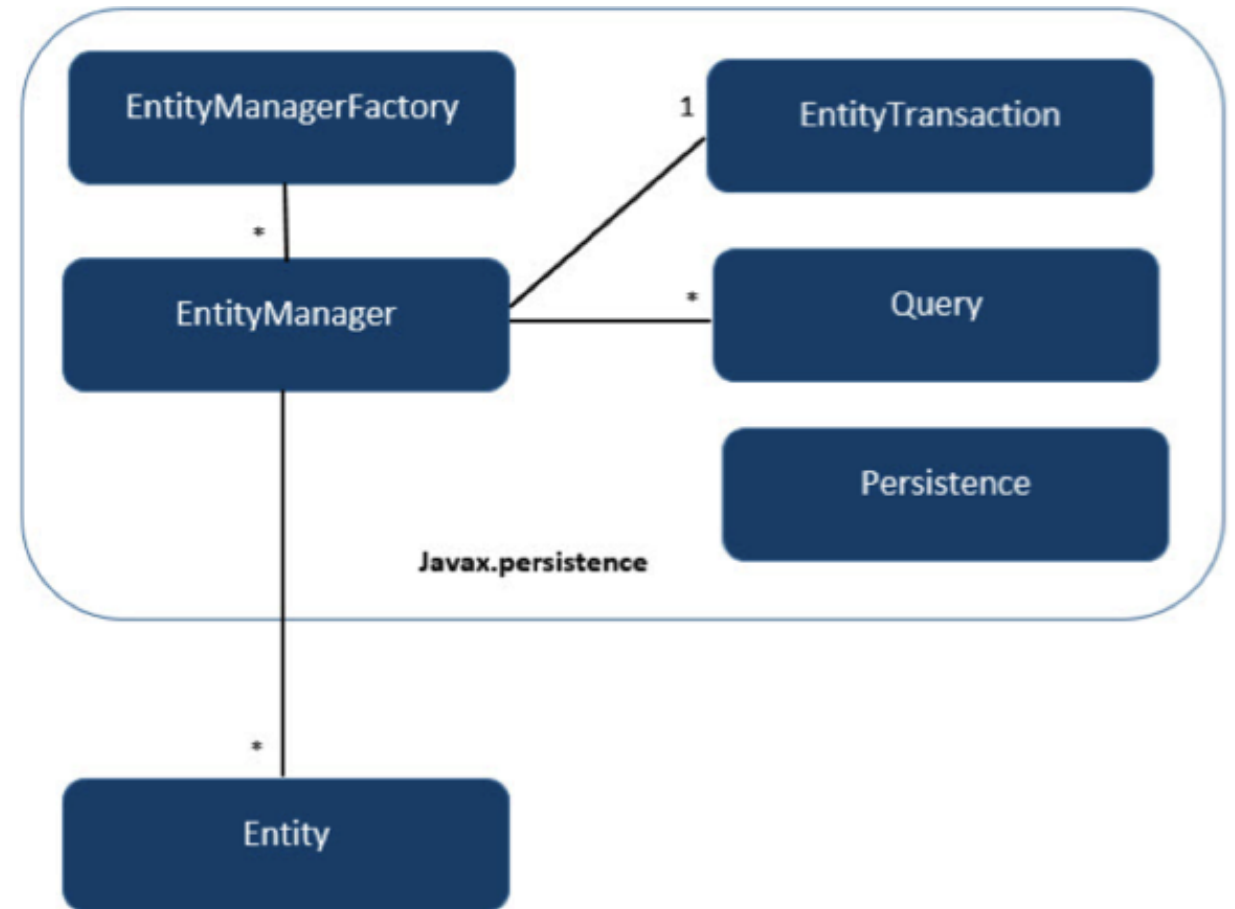
- Đơn giản hóa công nghệ cho tầng persistence (tầng dữ liệu).
- Không phụ thuộc vào các framework ORM.
- Có nhiều nhà cung cấp hỗ trợ cài đặt JPA.
- Dữ liệu có thể được lưu trữ thông qua việc ORM.

- **Tại sao nên dùng JPA:**

- Viết ít code hơn.
- Performance tốt.
- Độc lập về database.
- Không phải làm việc với SQL.
- Hỗ trợ cấu hình triển khai bằng annotation và xml.
- Có nhiều framework ORM miễn phí hỗ trợ có thể dùng để phát triển nhiều loại ứng dụng khác nhau.
- JPA là một đặc tả đã được chuẩn hóa và là một thành phần trong đặc tả EJB 3.
- Dễ dàng chuyển từ một ORM này sang một ORM khác.

| Kiến trúc và các thành phần của JPA

- **EntityManager**: interface cung cấp các API cho việc tương tác với các Entity.
- **EntityManagerFactory**: được dùng để tạo ra một instance của EntityManager.
- **Persistence**: định nghĩa một tập hợp các Entity class được quản lý bởi 1 instance của EntityManager trong ứng dụng.
- **EntityTransaction**: có quan hệ 1-1 với EntityManager. Bất kỳ thao tác nào được bắt đầu thông qua đối tượng **EntityManager** đều được đặt trong một Transaction.
- **Query**: là một interface, được nhà cung cấp JPA implement để có được các đối tượng quan hệ đáp ứng các tiêu chí truy vấn.



Ví dụ với Entity Bean

```
//EntityBean: BookEntity
package ejb.book;
@Entity
public class BookEntity implements Serializable {
    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    @Column(nullable = false)
    private String title;
    private float price;
    @Column(length = 2000)
    private String description;
    private String isbn;
    private int nbOfPage;
    private boolean illustrations;
    public BookEntity() {
    }
    public BookEntity(Long id, String title, float price, String
description, String isbn, int nbOfPage, boolean illustrations) {
        this.id = id;
        this.title = title;
        this.price = price;
        this.description = description;
        this.isbn = isbn;
        this.nbOfPage = nbOfPage;
        this.illustrations = illustrations;
    }
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getDescription() {
        return description;
    }
    public void setDescription(String description) {
        this.description = description;
    }
}
```

```
//Remote Interface: BookRemote
package ejb.book;
import java.util.List;
import javax.ejb.Remote;
@Remote
public interface BookRemote {
    void create(BookEntity book);
    void edit(BookEntity book);
    void remove(BookEntity book);
    BookEntity find(Object id);
    List<BookEntity> findAll();
    List<BookEntity> findRange(int[] range);
    int count();
}
```

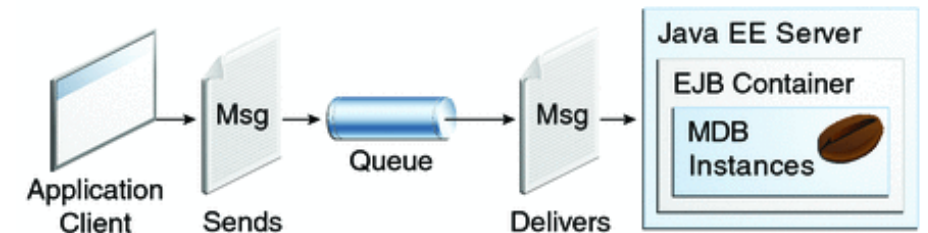
Ví dụ với Entity Bean

Sử dụng **EntityManager API** để tạo mới, cập nhật, xóa, truy vấn đối tượng.

```
//Stateless session bean: BookBean
@Stateless
public class BookBean implements BookRemote {
    @PersistenceContext(unitName = "BookWSPU")
    private EntityManager em;
    public void create(BookEntity book) {
        em.persist(book);
    }
    public void edit(BookEntity book) {
        em.merge(book);
    }
    public void remove(BookEntity book) {
        em.remove(em.merge(book));
    }
    public BookEntity find(Object id) {
        return em.find(BookEntity.class, id);
    }
    public List<BookEntity> findAll() {
        CriteriaQuery cq = em.getCriteriaBuilder().createQuery();
        cq.select(cq.from(BookEntity.class));
        return em.createQuery(cq).getResultList();
    }
    public List<BookEntity> findRange(int[] range) {
        CriteriaQuery cq = em.getCriteriaBuilder().createQuery();
        cq.select(cq.from(BookEntity.class));
        Query q = em.createQuery(cq);
        q.setMaxResults(range[1] - range[0]);
        q.setFirstResult(range[0]);
        return q.getResultList();
    }
    public int count() {
        CriteriaQuery cq = em.getCriteriaBuilder().createQuery();
        Root<BookEntity> rt = cq.from(BookEntity.class);
        cq.select(em.getCriteriaBuilder().count(rt));
        Query q = em.createQuery(cq);
        return ((Long) q.getSingleResult()).intValue();
    }
}
```

| Message Driven Bean

- Một bean **hướng thông điệp MDB** là một enterprise bean cho phép các ứng dụng Java EE xử lý các thông điệp theo cách không đồng bộ.
- Nhận và xử lý các Java Message Service (JMS).
- MDB có thể được xem như một JMS Consumer.
- MDB có thể làm việc theo chế độ một-một hay quảng bá kiểu một-nhiều.
- Một thành phần MDB hoạt động như sau:
 - Container đăng ký thành phần MDB này với JMS
 - JMS đăng kí tất cả các đích JMS với JNDI
 - EJB container thể hiện thành phần MDB này
 - Client tìm kiếm đích với MDB
 - Client gửi thông điệp đến đích
 - EJB container chọn MDB tương ứng để xử lý gói tin.



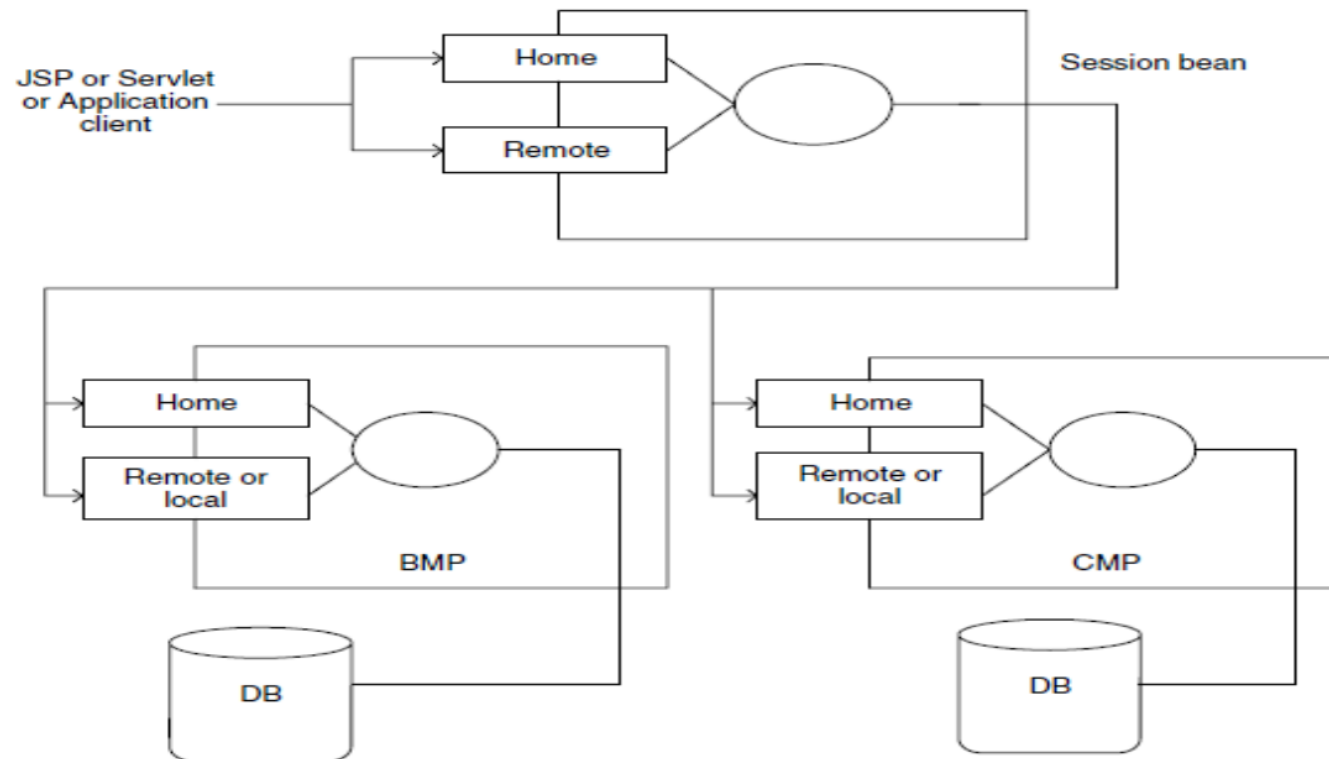
Ví dụ Message Driven Bean

Các gói tin được truyền và xử lý theo phương thức `MessageListener` được gọi là `onMessage()`.

```
//MessageDrivenBean: SimpleMessageBean
package ejb.mdb;
import javax.ejb.MessageDriven;
import javax.ejb.MessageDrivenContext;
import javax.jms.MessageListener;
import javax.jms.Message;
import javax.jms.TextMessage;
import javax.jms.JMSEException;
import javax.annotation.Resource;
@Resource(mappedName = "jms/Queue")
public class SimpleMessageBean implements MessageListener {
    @Resource
    private MessageDrivenContext mdc;
    public SimpleMessageBean() {
    }
    @Override
    public void onMessage(Message inMessage) {
        TextMessage msg = null;
        try {
            if (inMessage instanceof TextMessage) {
                msg = (TextMessage) inMessage;
                System.out.println("MESSAGE BEAN: Message received: " + msg.getText());
            } else {
                System.out.println("Message of wrong type: " + inMessage.getClass().getName());
            }
        } catch (JMSEException e) {
            e.printStackTrace();
            mdc.setRollbackOnly();
        } catch (Throwable te) {
            te.printStackTrace();
        }
    }
}
```

| 3.4 MÔ HÌNH KẾT NỐI CỦA EJB

- Có thể có các kết nối **đồng bộ** hoặc **không đồng bộ** giữa hai thành phần EJB thông qua giao diện **từ xa** hoặc **cục bộ** của các thành phần.
- Một thành phần EJB cũng có thể được truy nhập bằng thành phần Web, thành phần JavaBean, một thành phần Servlet, hoặc một thành phần JSP. Một thành phần EJB cũng có thể truy cập các đối tượng dữ liệu khác bằng JDBC.

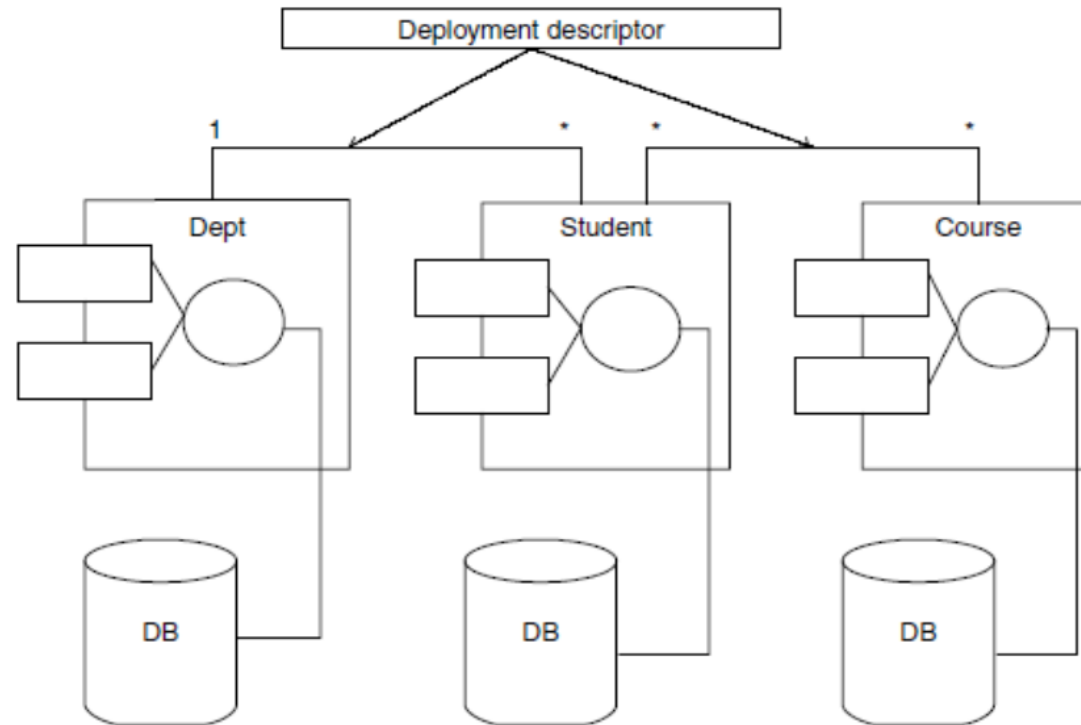


| KẾT HỢP CÁC KẾT NỐI GIỮA CÁC ENTITY BEAN

- Các quan hệ giữa các **Entity bean** thực thể có thể là một-một, một-nhiều hay nhiều-nhiều, giống như các quan hệ dữ liệu trong cơ sở dữ liệu được thể hiện trong biểu đồ quan hệ-thực thể.

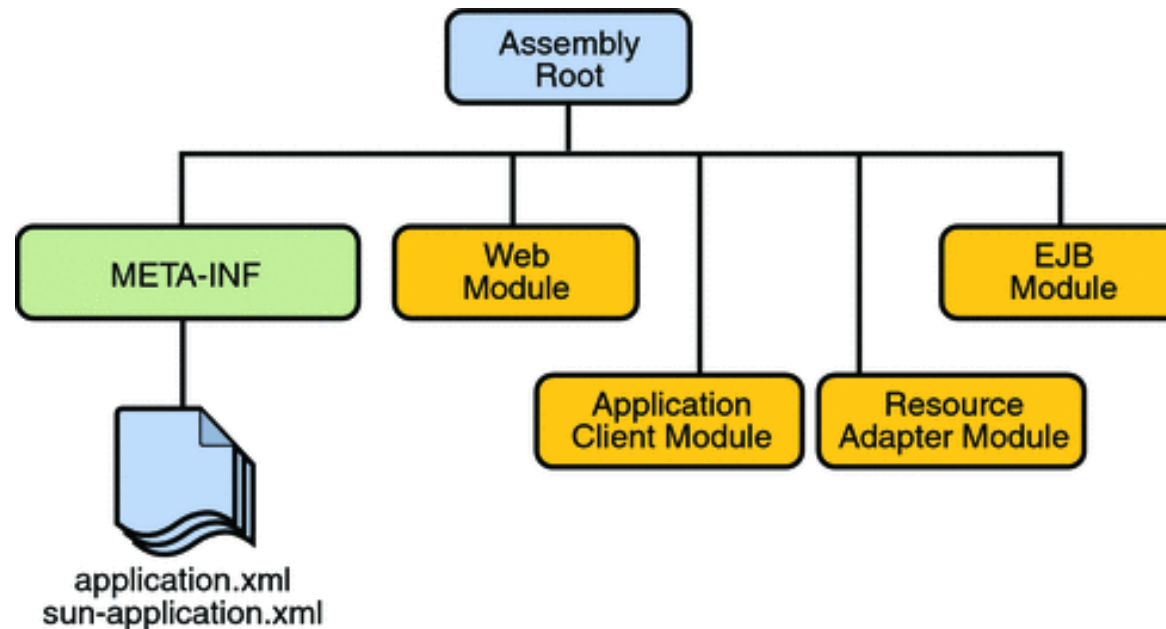
- Ví dụ:

Bean Dept có quan hệ một - nhiều với bean Student và bean Student có quan hệ nhiều – nhiều với bean Course.



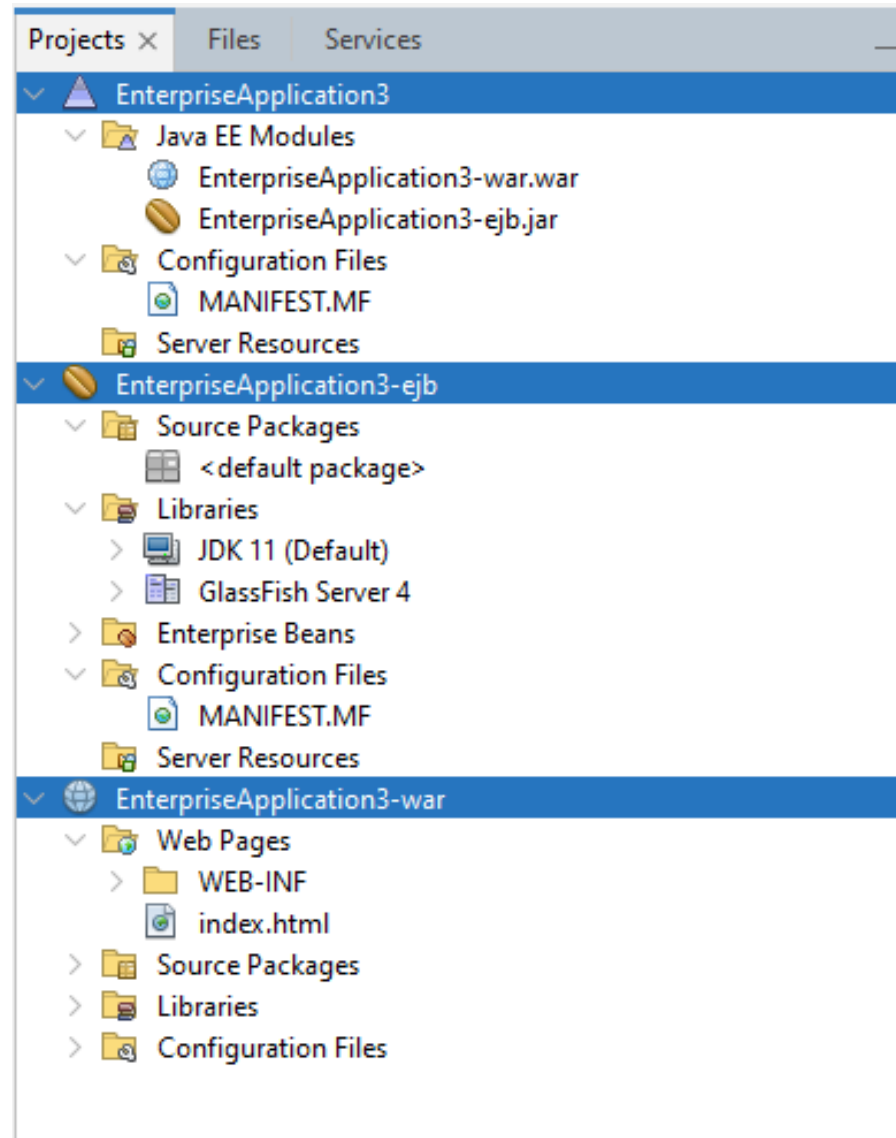
| 3.5 MÔ HÌNH TRIỂN KHAI EJB

- Một thành phần EJB được đóng gói thành file *.JAR, sau đó sẽ được tổ hợp với các gói Web thành phần *.WAR và các gói ứng dụng client thành file ứng dụng *.EAR (Enterprise Archive File) để triển khai trên máy chủ (Application Server).



Cấu trúc file EAR

| CẤU TRÚC DỰ ÁN ENTERPRISE APPLICATION



| JAVA APPLICATION SERVER

- Application Server:
 - Hiệu suất cao.
 - Bảo mật dữ liệu.
 - Tập trung cấu hình.
 - Toàn vẹn dữ liệu và mã nguồn.
 - Tính sẵn sàng sao.
- Java Application Server:
 - Apache Tomcat
 - Jetty
 - Glassfish
 - Apache Geronimo
 - JBoss
 - Web Logic.

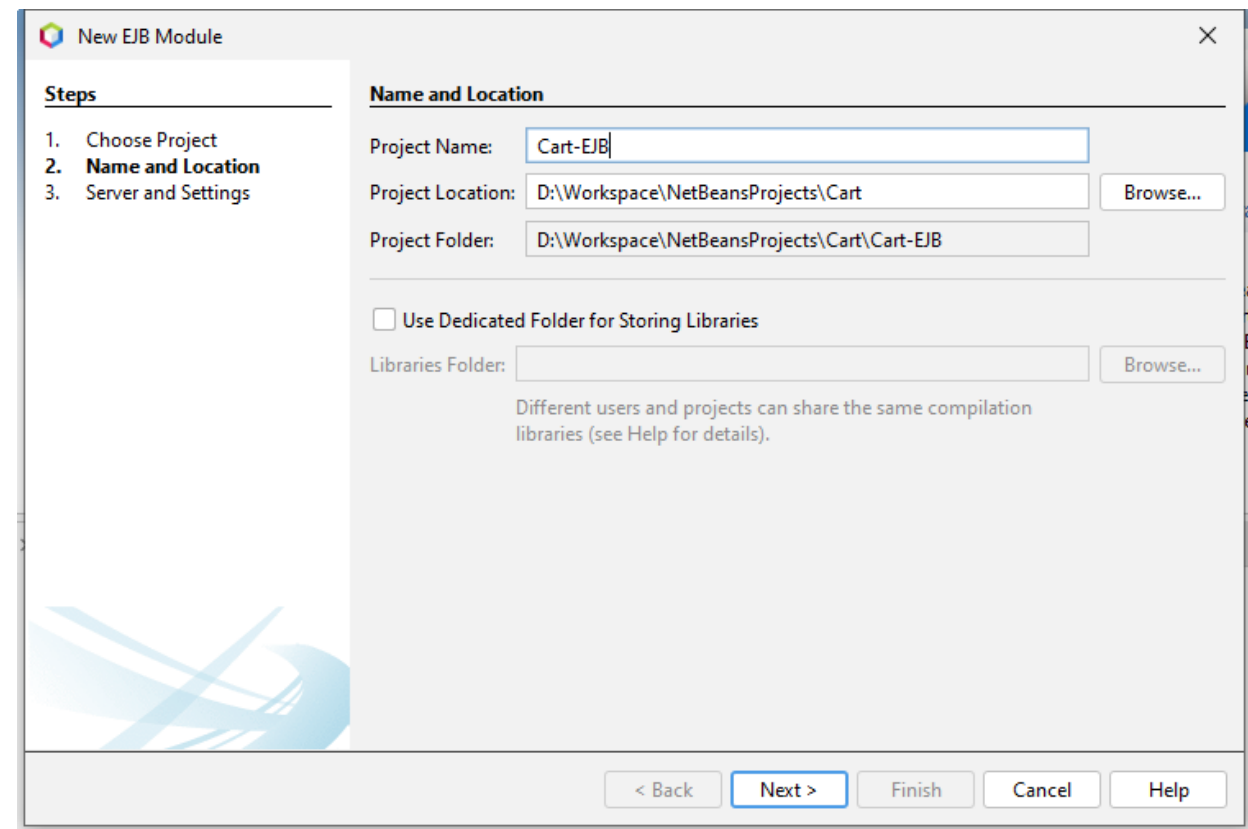
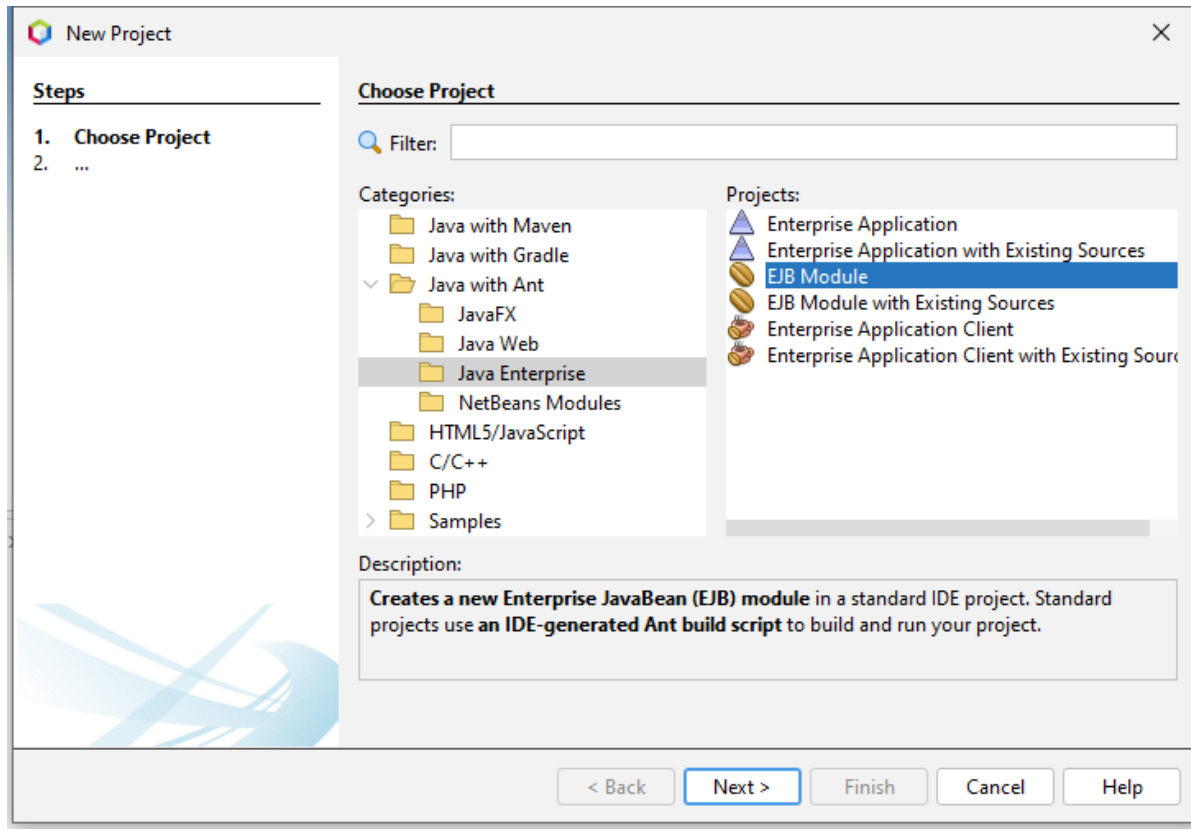
Web Servers	Application Servers
Servlets	Servlets
JSP	JSP
Expression Language	Expression Language
WebSocket	WebSocket
JTA	JTA
JASPIC	Batch
	CDI
	Bean Validation
	EJB
	JPA
	JMS
	JSON-P
	JavaMail
	JAX-RS
	JAX-WS

| 3.6 CASE STUDY

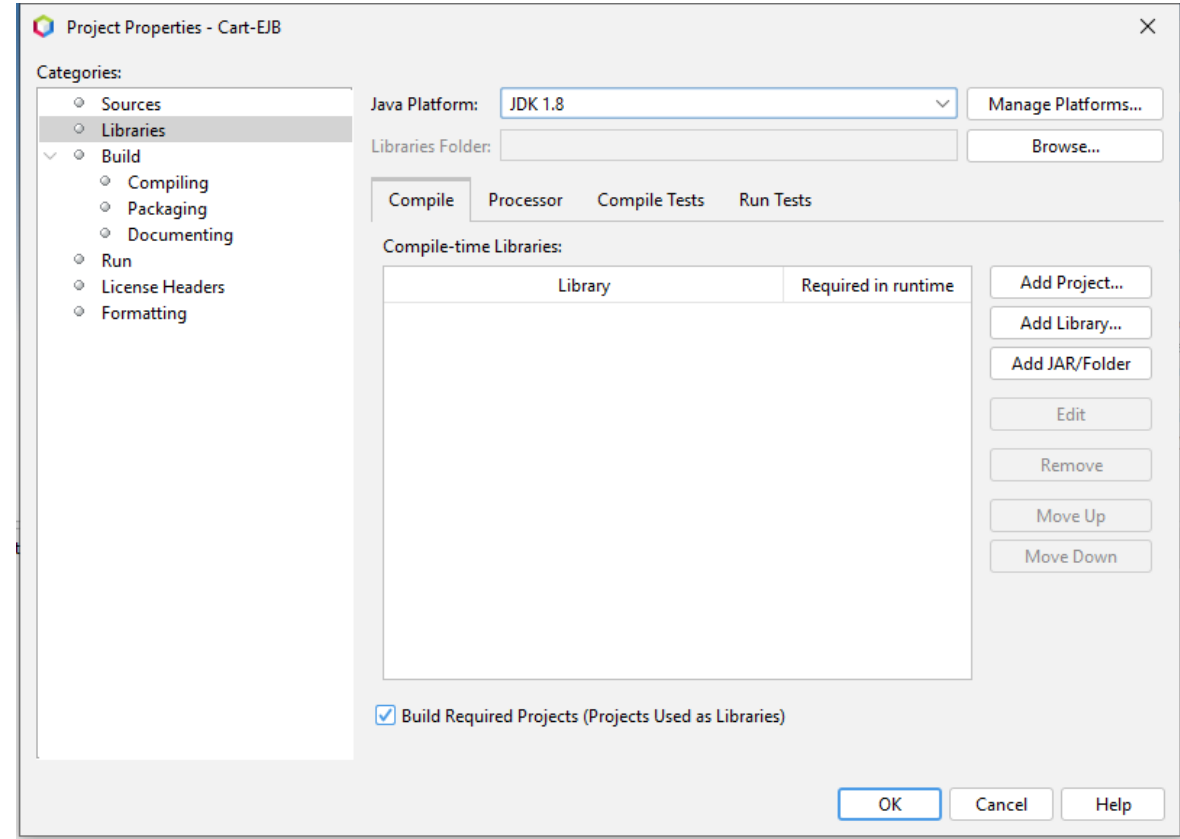
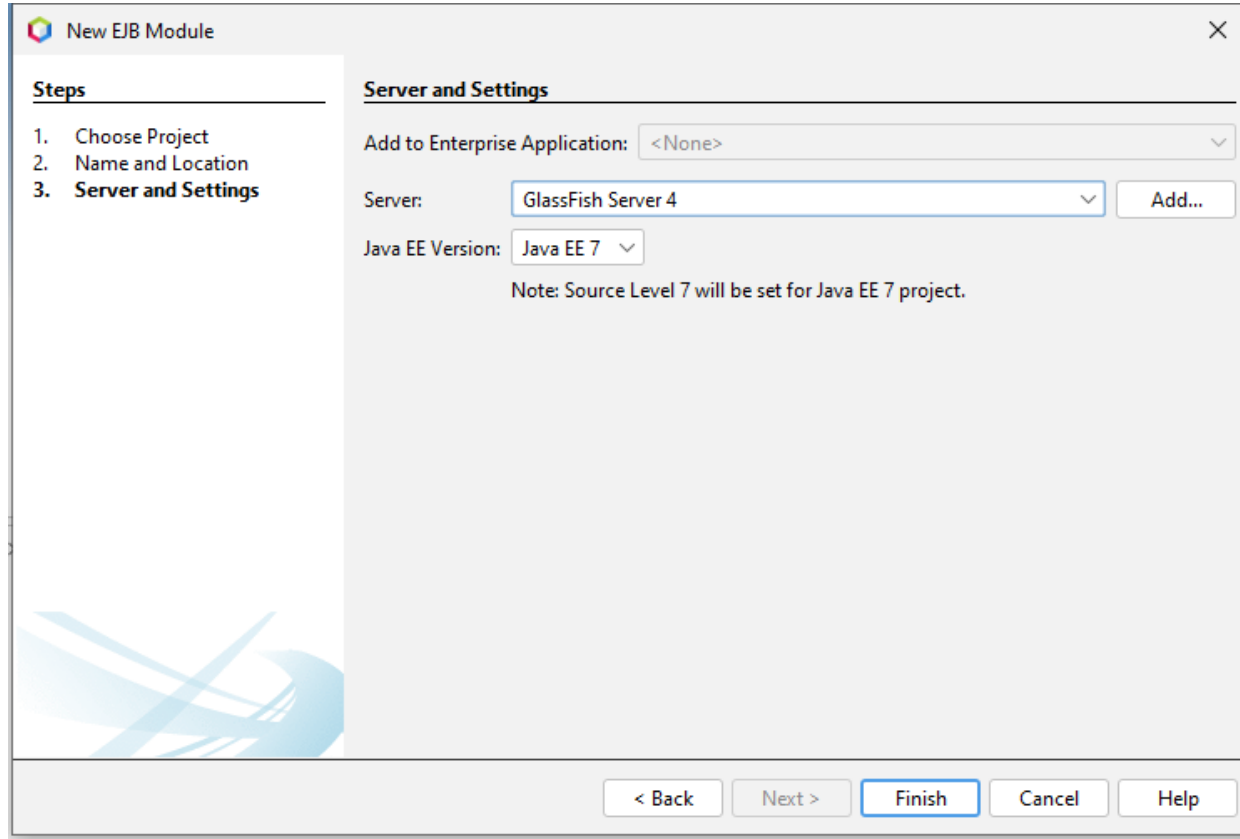
- Case Study 1: Biểu diễn một Shopping Cart trong Bookstore trực tuyến.
- Case Study 2: Xây dựng Converter với Netbean

| Case Study 1: Biểu diễn một Shopping Cart trong Bookstore trực tuyến

- **Bước 1:** Tạo Project EJB Module mới (phiên bản NetBeans IDE 20)



| Case Study 1: Biểu diễn một Shopping Cart trong Bookstore trực tuyến



- Chọn GlassFish Server 4.1, Java EE 7
- Java Platform JDK 1.8

| Case Study 1: Biểu diễn một Shopping Cart trong Bookstore trực tuyến

- Session Bean cart biểu diễn một shopping cart trong Bookstore trực tuyến. Client của bean có thể thêm, xóa sách ra khỏi giỏ, lấy lại nội dung của giỏ sách. Chúng ta cần xây dựng hai file:
 - Class CartBean
 - Interface nghiệp vụ
- **Bước 1:** Xây dựng Interface nghiệp vụ

```
package cart.ejb;

import cart.util.BookException;
import java.util.List;
import javax.ejb.Remote;

@Remote
public interface Cart {
    public void initialize(String person) throws BookException;
    public void initialize(String person, String id) throws BookException;
    public void addBook(String title);
    public void removeBook(String title) throws BookException;
    public List<String> getContents();
    public void remove();
}
```

| Case Study 1

- Tạo lớp bắt ngoại lệ BookException và IdVerifier:

```
package cart.util;  
  
public class BookException extends Exception {  
    public BookException() {  
    }  
    public BookException(String msg) {  
        super(msg);  
    }  
}
```

```
package cart.util;  
  
public class IdVerifier {  
    public boolean validate(String id) {  
        return true;  
    }  
}
```

- **Bước 2:** Xây dựng lớp Bean, CartBean phải đạt được các yêu cầu sau:
 - Lớp phải được chú thích @Stateful
 - Lớp cài đặt các phương thức định nghĩa trong interface nghiệp vụ

| Case Study 1

```
package cart.ejb;

import cart.util.BookException;
import cart.util.IdVerifier;
import java.util.ArrayList;
import java.util.List;
import javax.ejb.Remove;
import javax.ejb.Stateful;

@Stateful()
public class CartBean implements Cart {
    List<String> contents;
    String customerId;
    String customerName;

    public void initialize(String person) throws BookException {
        if (person == null) {
            throw new BookException("Null person not allowed.");
        } else {
            customerName = person;
        }
        customerId = "0";
        contents = new ArrayList<String>();
    }

    public void initialize(String person, String id) throws BookException {
        if (person == null) {
            throw new BookException("Null person not allowed.");
        } else {
            customerName = person;
        }
        IdVerifier idChecker = new IdVerifier();
        if (idChecker.validate(id)) {
            customerId = id;
        } else {
            throw new BookException("Invalid id: " + id);
        }
        contents = new ArrayList<String>();
    }
}
```

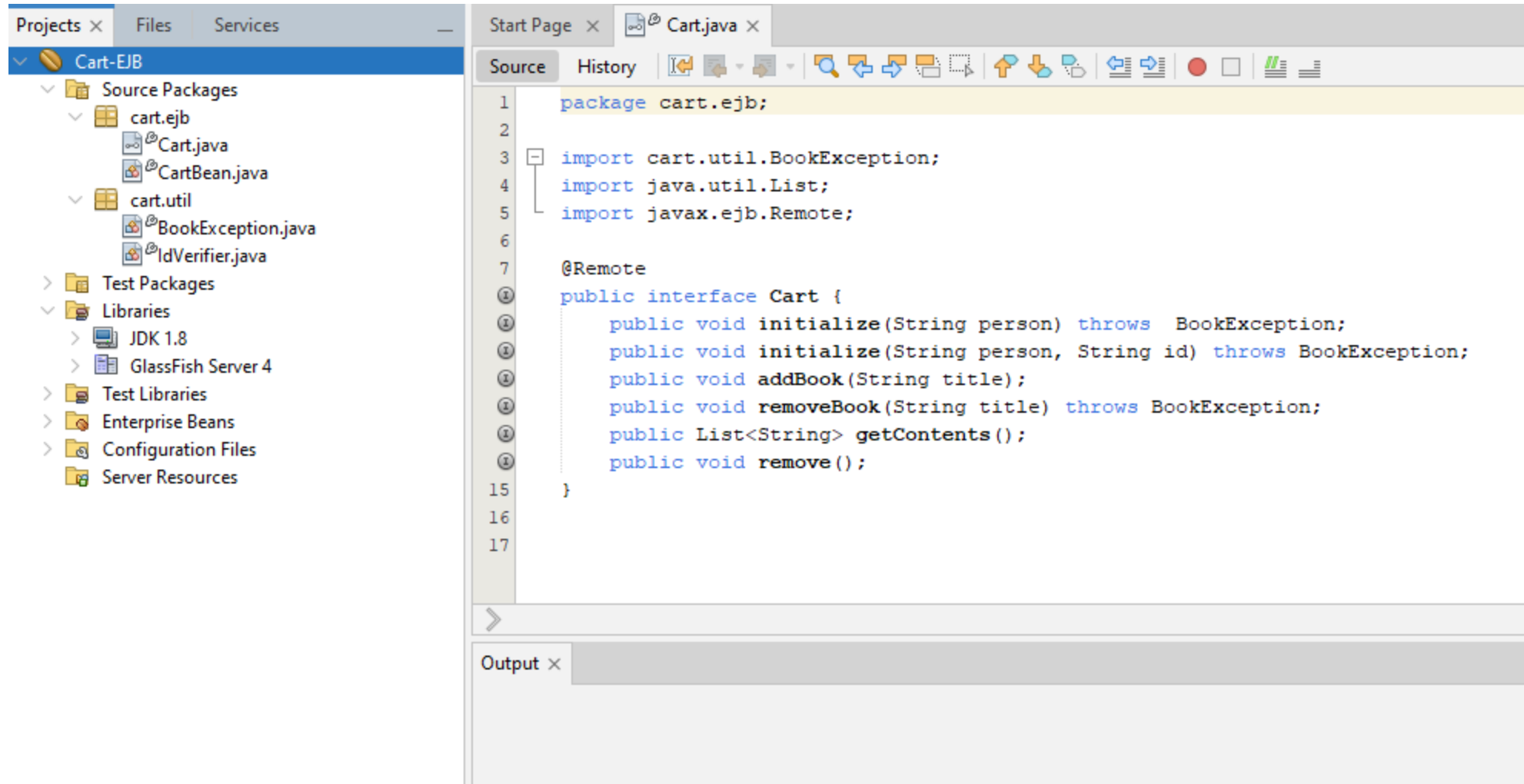
```
public void addBook(String title) {
    contents.add(title);
}

public void removeBook(String title) throws BookException {
    boolean result = contents.remove(title);
    if (result == false) {
        throw new BookException("\"" + title + "\" not in cart.");
    }
}

public List<String> getContents() {
    return contents;
}

@Remove()
public void remove() {
    contents = null;
}
}
```

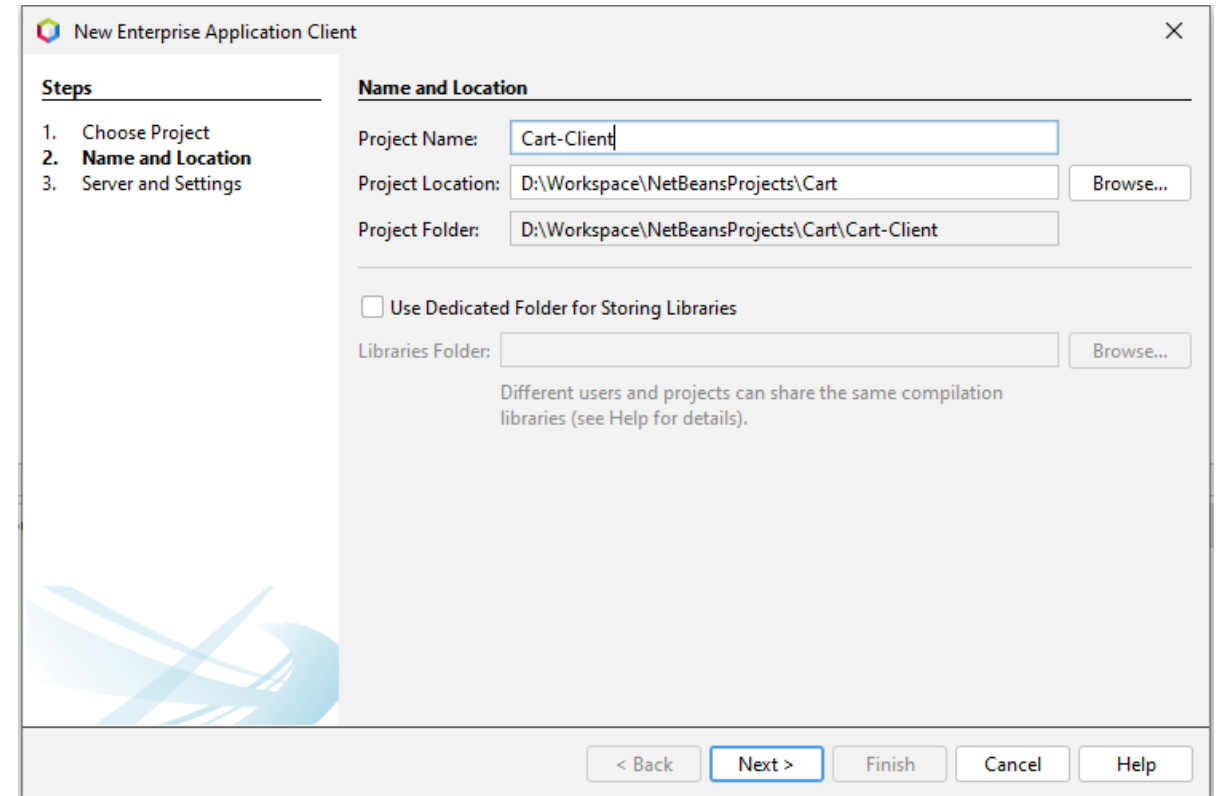
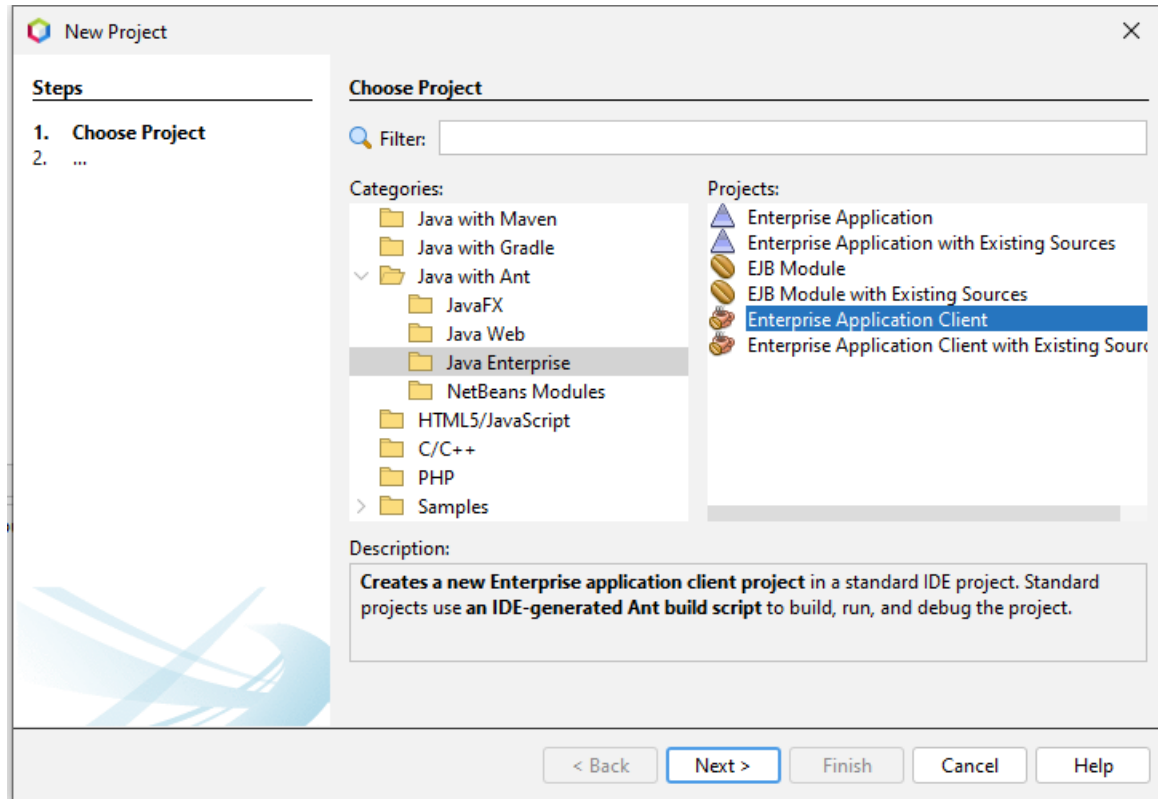
| Case Study 1



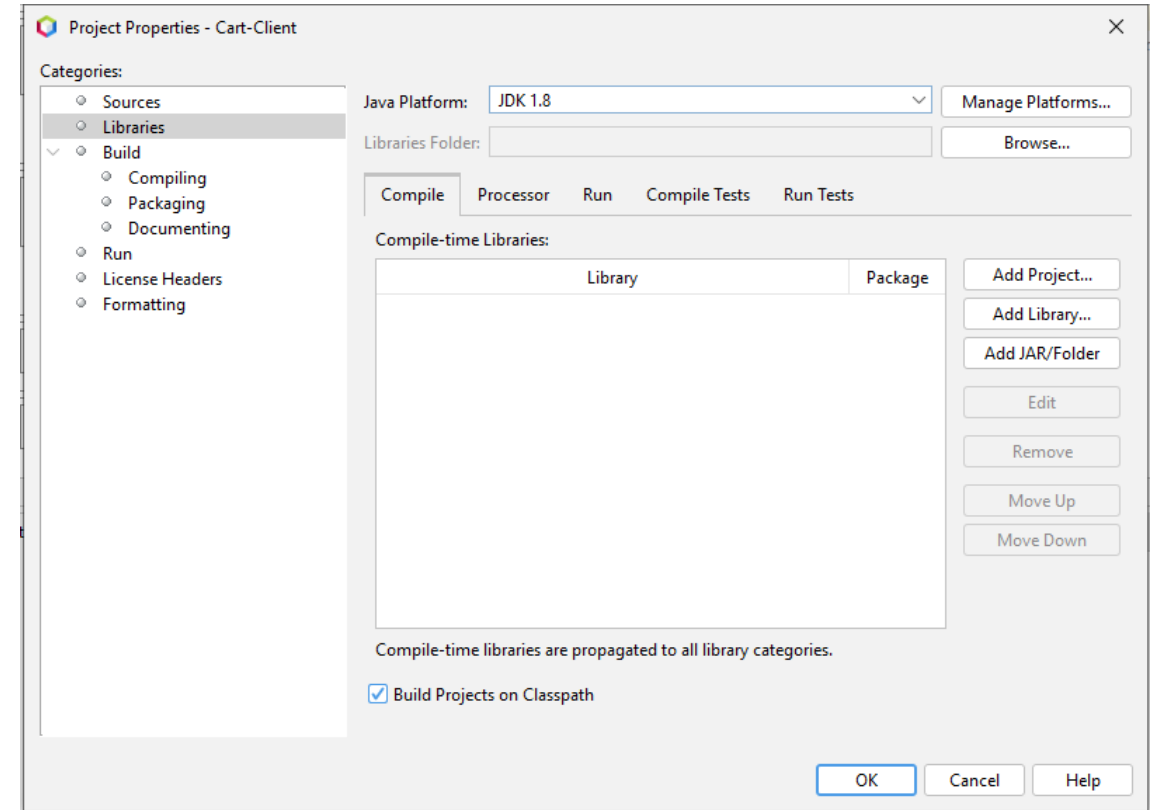
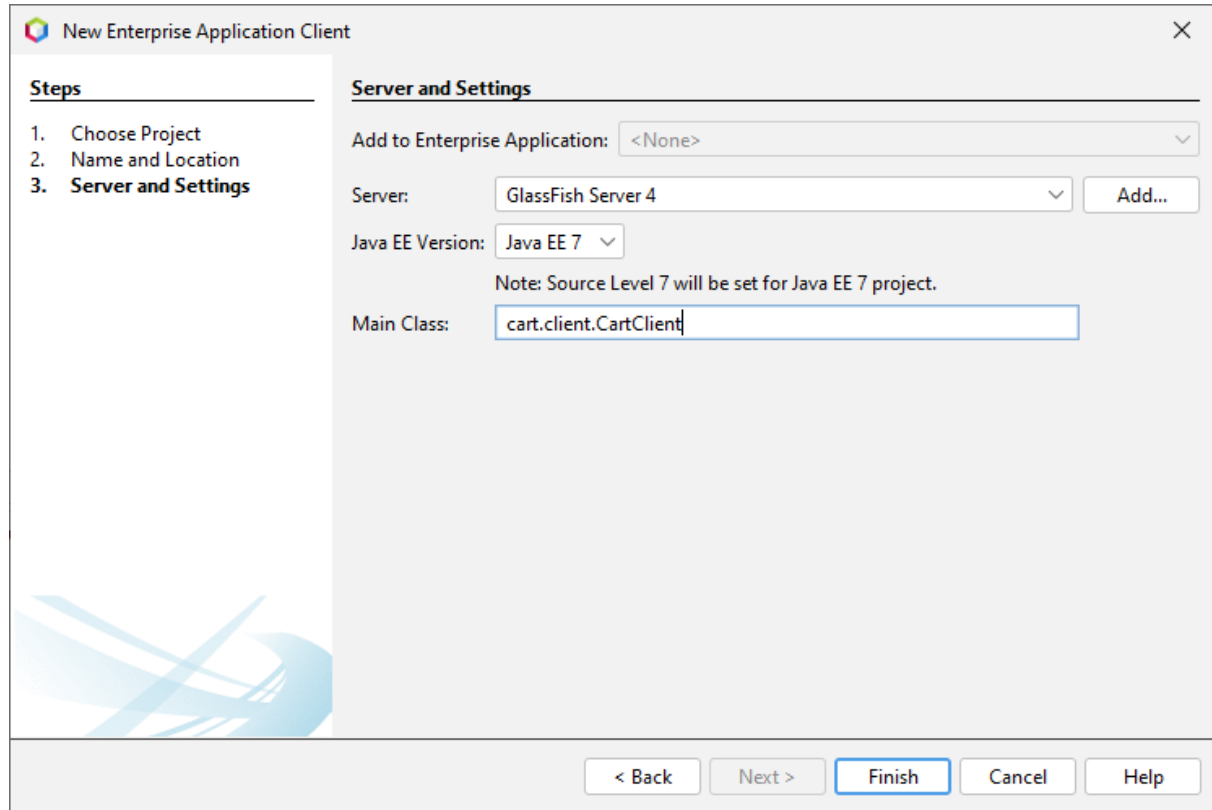
Cấu trúc Cart-EJB Project.

| Case Study 1

• Bước 3: Xây dựng Client, Tạo Project Enterprise Application Client

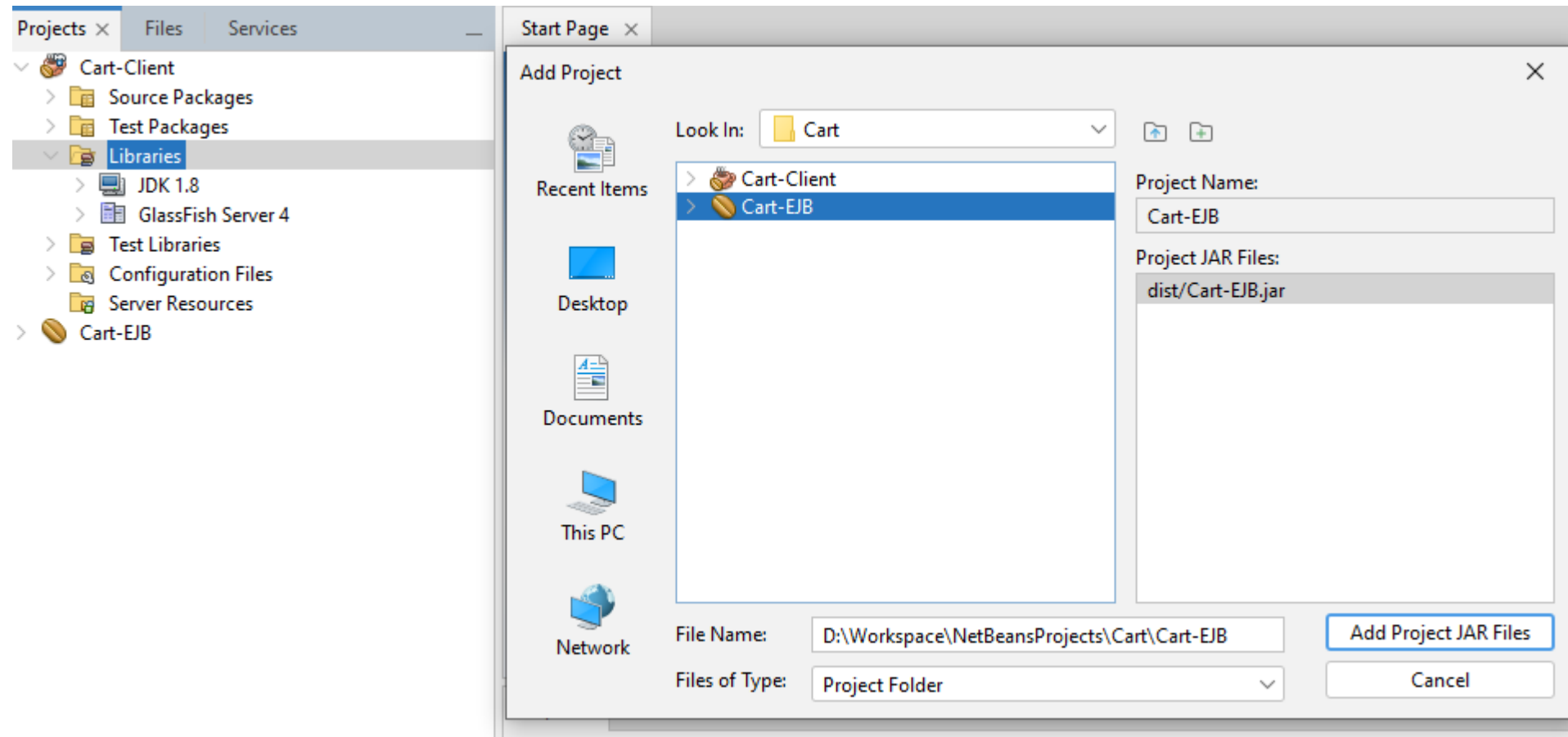


| Case Study 1



- Chọn GlassFish Server 4.1, Java EE 7
- Java Platform JDK 1.8

| Case Study 1



- Add Project Cart-EJB vào Libraries của Cart-Client

Case Study 1

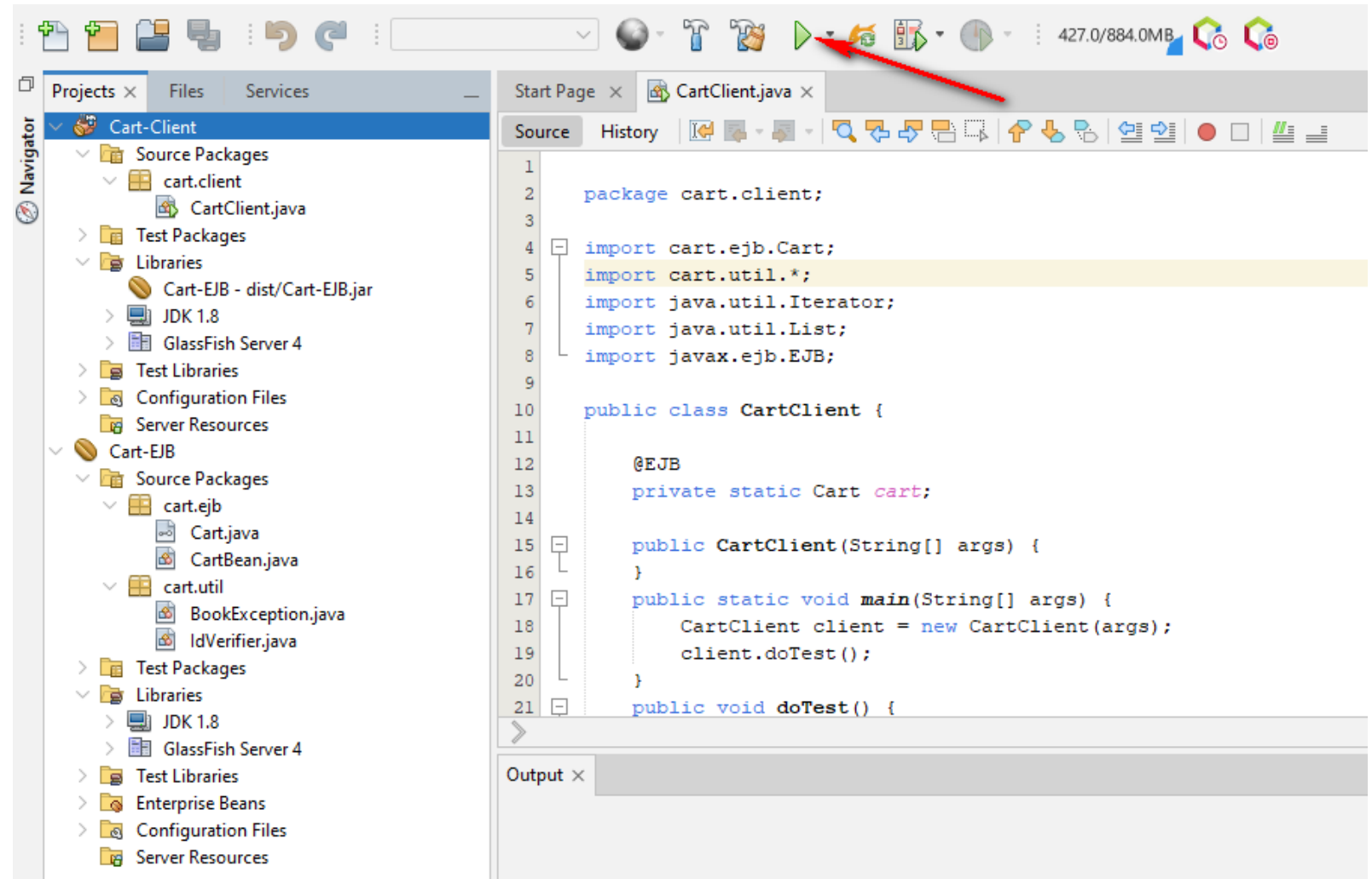
- Bước 3: Xây dựng Client.

```
package cart.client;
import cart.ejb.Cart;
import cart.util.*;
import javax.ejb.EJB;

public class CartClient {
    @EJB
    private static Cart cart;
    public CartClient(String[] args) {
    }
    public static void main(String[] args) {
        CartClient client = new CartClient(args);
        client.doTest();
    }
    public void doTest() {
        try {
            cart.initialize("Duke d'Url", "123");
            cart.addBook("Infinite Jest");
            cart.addBook("Bel Canto");
            cart.addBook("Kafka on the Shore");
            List<String> bookList = cart.getContents();
            bookList = cart.getContents();
            Iterator<String> iterator = bookList.iterator();
            while (iterator.hasNext()) {
                String title = iterator.next();
                System.out.println("Retrieving book title from cart: " + title);
            }
            System.out.println("Removing \"Gravity's Rainbow\" from cart.");
            cart.removeBook("Gravity's Rainbow");
            cart.remove();
            System.exit(0);
        } catch (BookException ex) {
            System.err.println("Caught a BookException: " + ex.getMessage());
            System.exit(1);
        } catch (Exception ex) {
            System.err.println("Caught an unexpected exception!");
            ex.printStackTrace();
            System.exit(1);
        }
    }
}
```

Case Study 1

- Right Click Cart-EJB -> Deploy
- Run Project Cart-Client



| Case Study 1 – Kết quả

The screenshot displays the NetBeans IDE interface. On the left, the 'Projects' pane shows a project named 'Cart-Client' with sub-packages 'cart.client' and 'Test Packages'. The 'Sources' pane shows the 'CartClient.java' file. The main editor window displays the source code of 'CartClient.java'.

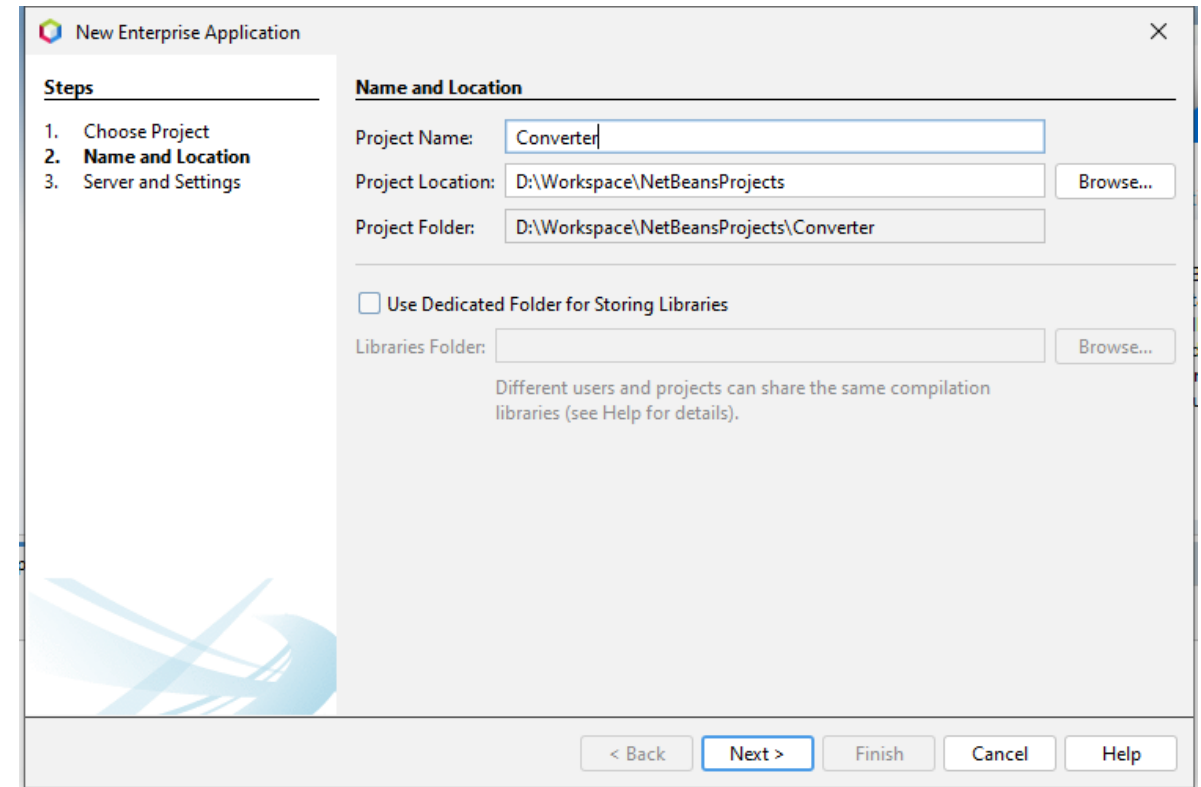
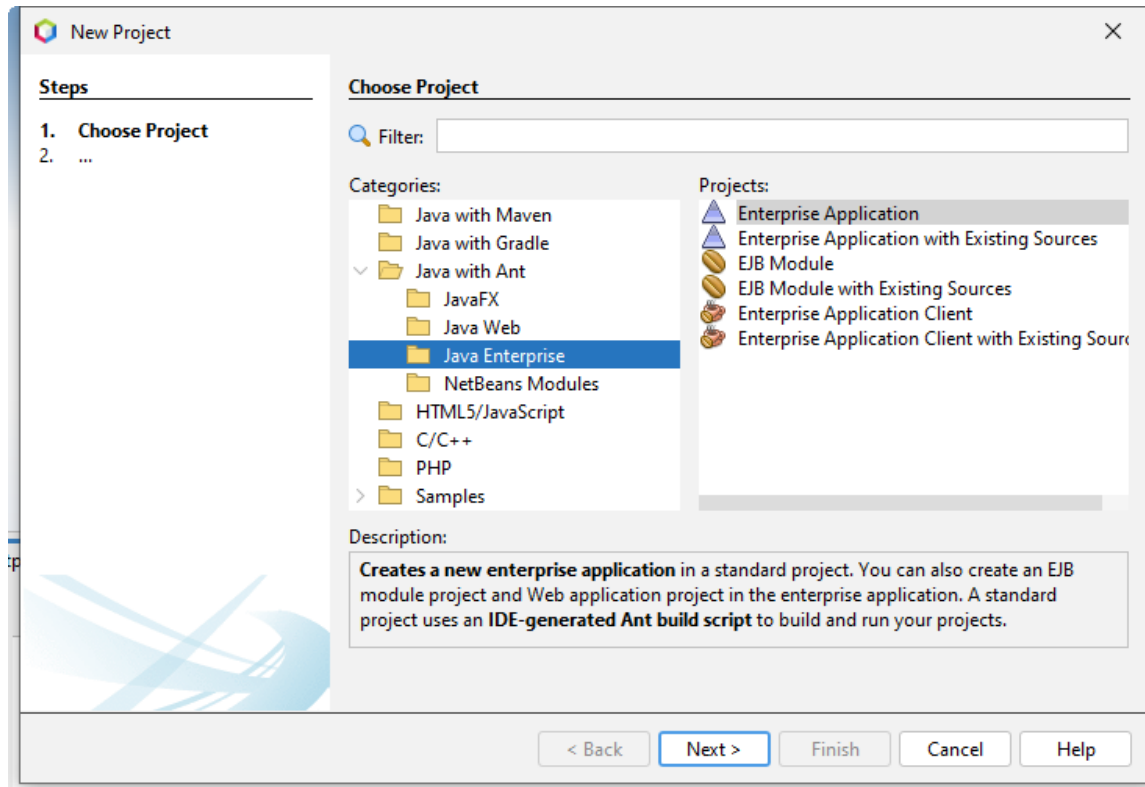
```
1 package cart.client;
2
3
4 import cart.ejb.Cart;
5 import cart.util.*;
6 import java.util.Iterator;
7 import java.util.List;
8 import javax.ejb.EJB;
9
10 public class CartClient {
11
12     @EJB
13     private static Cart cart;
14
15     public CartClient(String[] args) {
16     }
17
18     public static void main(String[] args) {
```

The 'Output' window at the bottom shows the execution results. It includes a warning about a missing directory and a list of book titles retrieved from the cart. The output is as follows:

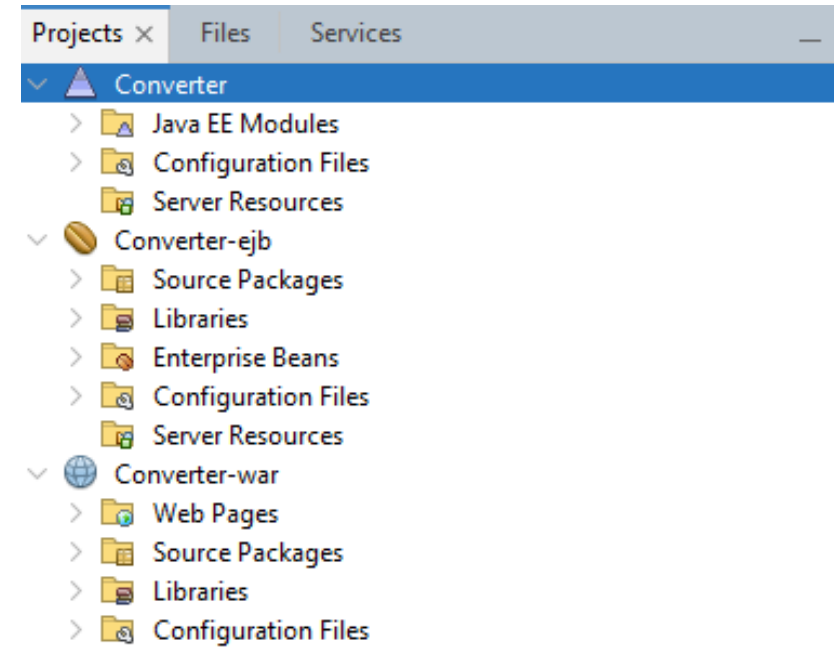
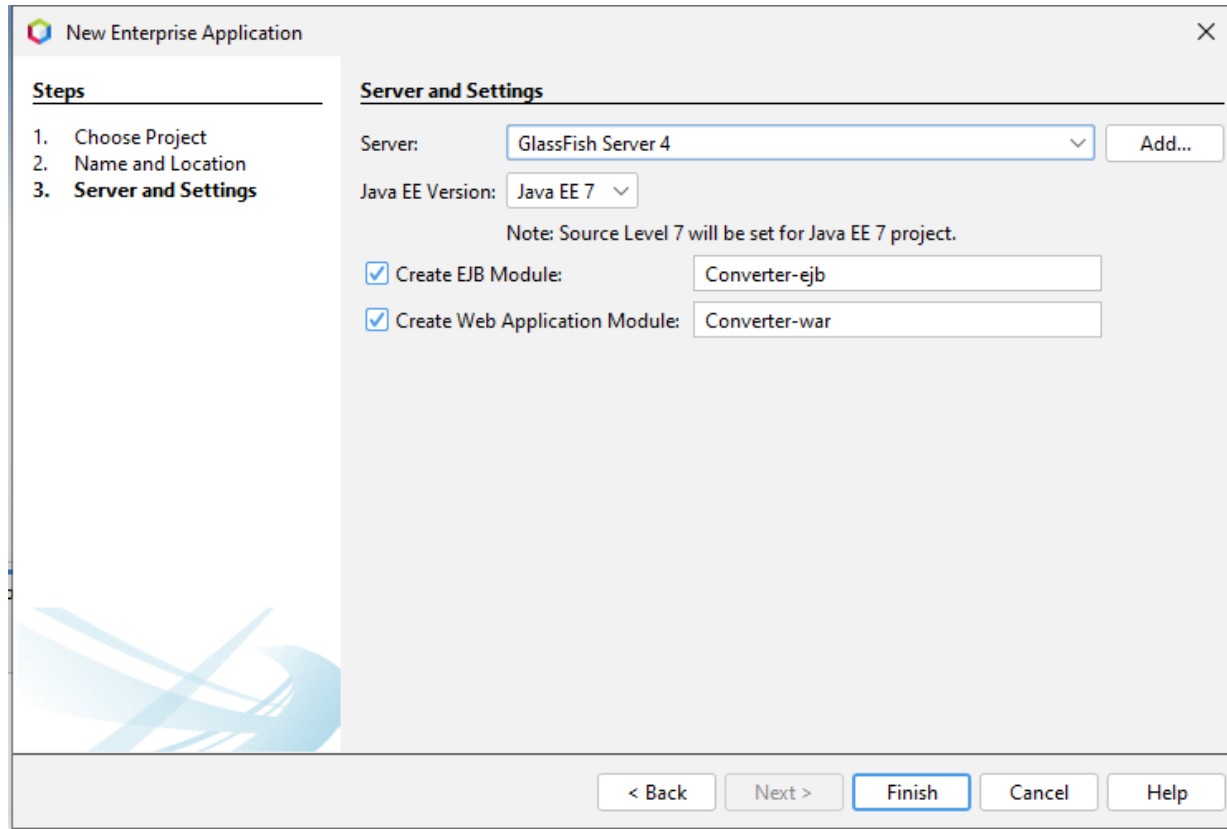
```
Java DB Database Process x GlassFish Server 4 x Cart-Client (run) x
Redeploying D:\Workspace\NetBeansProjects\Cart\Cart-Client\dist\Cart-Client.jar
post-run-deploy:
run-deploy:
Copying 1 file to D:\Workspace\NetBeansProjects\Cart\Cart-Client\dist
Copying 2 files to D:\Workspace\NetBeansProjects\Cart\Cart-Client\dist\Cart-ClientClient
Warning: D:\Workspace\NetBeansProjects\Cart\Cart-Client\dist\gfdeploy\Cart-Client does not exist.
Retrieving book title from cart: Infinite Jest
Retrieving book title from cart: Bel Canto
Retrieving book title from cart: Kafka on the Shore
Removing "Gravity's Rainbow" from cart.
Caught a BookException: "Gravity's Rainbow" not in cart.
Java Result: 1
run:
BUILD SUCCESSFUL (total time: 22 seconds)
```

| Case Study 2 - Xây dựng Converter với Netbeans

- **Ví dụ:** Sử dụng EJB tạo Converter chuyển đổi từ độ F sang độ C và ngược lại dựa trên Netbeans.
- **Bước 1:** Tạo Project mới (phiên bản NetBeans IDE 20)



| Case Study 2 - Xây dựng Converter với Netbeans



| Case Study 2 - Xây dựng Converter với Netbeans

- **Bước 2:** Tạo Converter Interface và ConverterBean trong Converter-ejb.

```
package converter.ejb;

import javax.ejb.Remote;

@Remote
public interface Converter {
    public double cToF(double c);
    public double fToC(double f);
}
```

```
package converter.ejb;

import javax.ejb.Stateless;

@Stateless
public class ConverterBean implements Converter {
    @Override
    public double cToF(double c) {
        return c * 9 / 5 + 32;
    }
    @Override
    public double fToC(double f) {
        return (f - 32) * 5 / 9;
    }
}
```

| Case Study 2 - Xây dựng Converter với Netbeans

- Bước 3: Viết trang `index.jsp` trong Converter-war

```
<%@ page import="converter.ejb.Converter, javax.naming.*"%>
<%@ page import="java.math.*"%>
<%@ page import="javax.naming.*"%>
<%@ page import="javax.ejb.*" %>
<%@ page import="java.text.DecimalFormat" %>
<%@ page import="java.rmi.RemoteException" %>
<%!
    private Converter converter = null;
    public void jspInit() {
        try {
            InitialContext ic = new InitialContext();
            converter = (Converter) ic.lookup(Converter.class.getName());
        } catch (Exception ex) {
            System.out.println("Couldn't create converter bean." + ex.getMessage());
        }
    }
    public void jspDestroy() {
        converter = null;
    }
%>
```

| Case Study 2 - Xây dựng Converter với Netbeans

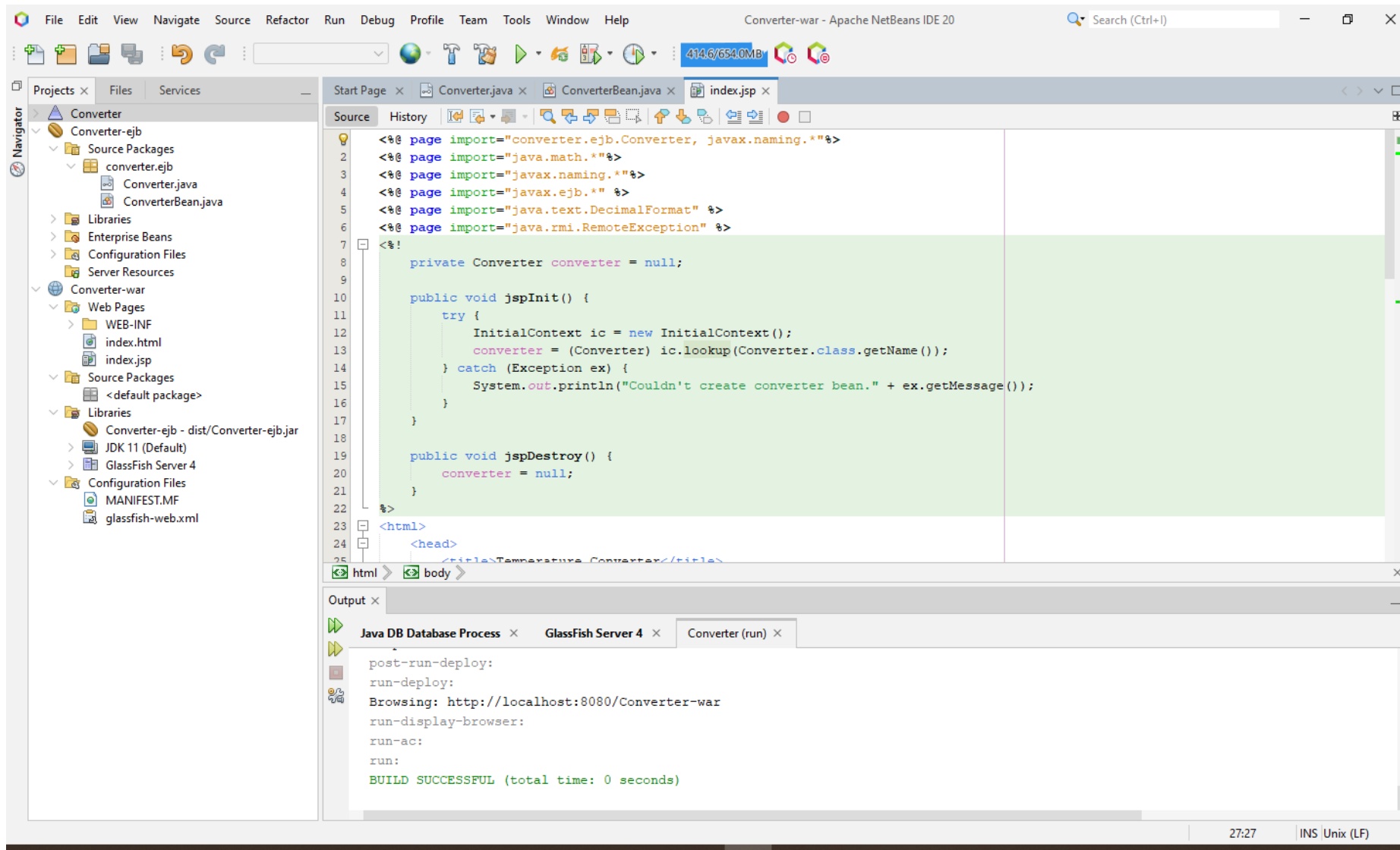
- Bước 3: Viết trang `index.jsp` trong Project `Converter-war`

```
<html>
  <head>
    <title>Temperature Converter</title>
  </head>
  <body bgcolor="white">
    <h1><b><center> Temperature Converter </center></b></h1>
    <hr>
    <p>Enter a degree to convert:</p>
    <form method="get">
      <input type="text" name="degree" size="25">
      <br><p>
        <input type="submit" name="fToC" value="Fahrenheit to Centigrade">
        <input type="submit" name="cToF" value="Centigrade to Fahrenheit">
      </form>
      <%
        DecimalFormat twoDigits = new DecimalFormat("0.00");
        String degree = request.getParameter("degree");
        if (degree != null && degree.length() > 0) {
          double d = Double.parseDouble(degree);

          if (request.getParameter("fToC") != null) {
            <p>
              <%= degree%> fahrenheit degrees are <%= twoDigits.format(converter.fToC(d))%> centigrade degrees.
            </p>
            if (request.getParameter("cToF") != null) {
              <p>
                <%= degree%> centigrade degrees are
                <%= twoDigits.format(converter.cToF(d))%> fahrenheit degrees .
              </p>
            }
          }
        }
      <%>
    </body>
  </html>
```

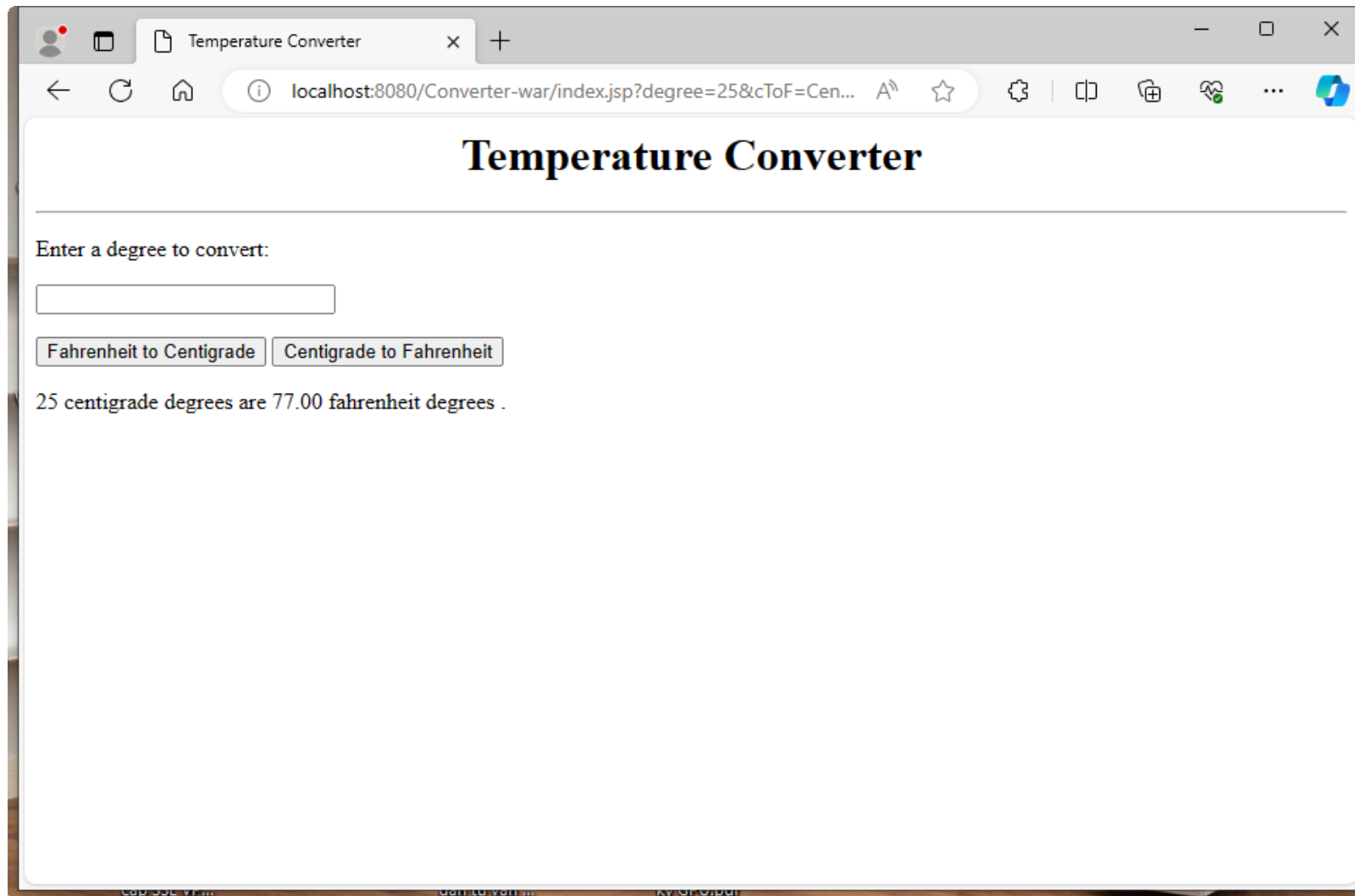
Case Study 2 - Xây dựng Converter với Netbeans

- Kết quả cây dự án:



| Case Study 2 - Xây dựng Converter với Netbeans

- Run Project Converter, Mở link <http://localhost:8080/Converter-war/index.jsp>



| BÀI TẬP

- Trình bày mô hình thành phần của của J2EE.
- Phân biệt Javabeans và EJB, ý nghĩa sử dụng của hai loại này.
- Cài đặt Netbeans hoặc Eclipse, thực hành Case Study 1 và 2.
- Tìm hiểu quy trình triển khai ứng dụng Enterprise Application trên Application Server.

| BÀI TẬP NHÓM – TÍNH ĐIỂM BÀI TẬP

Quy định:

- Nhóm từ 1 - 3 bạn.
- Nội dung: Mô hình hóa với UML và Xây dựng module chức năng với EJB.
- Hạn nộp bài: 29/03/2025 (In, đóng cuốn)

Nhóm chọn 1 trong các Bài tập sau hoặc Có thể chọn 1 Hệ thống khác:

- Xây dựng module quản lý mua hàng trong Hệ quản lý bán sách online với mô hình EJB.
- Xây dựng module quản lý mượn sách trong Hệ quản lý thư viện với mô hình thành phần EJB.
- Xây dựng module quản lý cho thuê xe ô tô với mô hình thành phần EJB.
- Xây dựng module quản lý đăng ký học tín chỉ với mô hình thành phần EJB.
- Xây dựng module quản lý nghe nhạc online với mô hình thành phần EJB.
- Xây dựng module quản lý đăng ký tour du lịch online với mô hình thành phần EJB.
- Xây dựng module quản lý đăng ký mua vé máy bay online với mô hình thành phần EJB.